

- Quantifying SZZ-Induced Label Noise in Just-In-Time Software Defect Prediction
 - Interim Report — Phases 1 and 2
 - Abstract
 - Abbreviations
 - A note on corrections
- Chapter 1 — Introduction
 - 1.1 Context
 - 1.2 The ground-truth problem
 - 1.3 The realism problem
 - 1.4 Problem statement
 - 1.5 Research gap
 - 1.6 Research questions
 - 1.7 Contributions
 - 1.8 Thesis organisation
- Chapter 2 — Background and Related Work
 - 2.1 Just-in-time software defect prediction
 - 2.2 SZZ and its variants
 - 2.3 Evaluation methodology in defect prediction
 - 2.4 Online learning for JIT-SDP
 - 2.5 The weak-baseline problem
 - 2.6 Metric selection
 - 2.7 Learning with noisy labels
 - 2.8 Positioning
 - 2.9 Citation verification status
- Chapter 3 — Research Design
 - 3.1 The phase pipeline
 - 3.2 Data
 - 3.3 SZZ variant implementations
 - 3.4 Metrics and statistical protocol
 - 3.5 Threats to validity, framed in advance
 - 3.6 Reproducibility
- Chapter 4 — Phase 1: Label Disagreement and Oracle Comparison
 - 4.1 Method
 - 4.2 Results
 - 4.3 The bias model
 - 4.4 Discussion — answering RQ1
- Chapter 5 — Phase 2: Downstream Impact Under Honest Evaluation
 - 5.1 Method
 - 5.2 Results

- 5.3 An honest negative—
- 5.3 An honest negative: the batch-to-stream contrast could not be isolated
- 5.3b An exploratory finding: the JITLine anomaly
- 5.4 Robustness
- 5.5 Discussion — answering RQ2
- Chapter 6 — Conclusions and Planned Work
 - 6.1 What Phases 1 and 2 establish
 - 6.2 What Phases 1 and 2 do not establish
 - 6.3 Commitments made in advance
 - 6.4 Threats to validity
 - 6.5 Closing
- References

Quantifying SZZ-Induced Label Noise in Just-In-Time Software Defect Prediction

Interim Report — Phases 1 and 2

Puneet Deshwani — M.Tech

Abstract

Just-in-time software defect prediction (JIT-SDP) labels almost universally descend from the SZZ algorithm, which infers defect-introducing commits by blaming the lines a bug fix modifies. That inference fails whenever bug-fixing commits are tangled, and the field has no measurement of what the resulting noise costs a model under realistic deployment conditions — in part because models are conventionally evaluated against the same SZZ labels that trained them.

This report covers the first two phases of that measurement, across 21 Apache Java projects and 27,319 commits, holding out an independently constructed reference label set so that a model trained on SZZ can be scored against something other than SZZ.

Phase 1 — label quality. Six SZZ variants are scored over an identical denominator. No variant exceeds **27.2% precision**. Noise is class-conditional rather than symmetric (false-alarm rates 6.7–26.3%, miss rates 35.9–73.3%). Variants agree with one another at κ up to **0.933** while agreeing with the reference at κ **0.158–0.202** — they share failure modes, so mutual agreement is not evidence of correctness. Decomposing the misses shows that for the most aggressive variants, **more than half of what their filters remove is correct**: refinement has relocated error rather than reduced it.

Phase 2 — downstream impact. Three models across seven label sources and four evaluation regimes, 16,380 runs, tested by paired Wilcoxon at the project level with Hodges–Lehmann estimates, bootstrap intervals and Holm correction applied both within family and globally. Random k -fold cross-validation inflates a high-capacity model by **+0.137 MCC** (21 of 21 projects); scoring against the training labels inflates it by a further **+0.230**. The two protocol

effects together are roughly **nine times** the label-source effect, and both scale with model capacity. Under honest streaming evaluation with reconstructed verification latency, training on the reference labels beats all six SZZ variants, five of six surviving global correction.

A result reading **0.41 MCC** under the field’s standard configuration is **0.10** under a defensible one. Because self-scoring is only possible where labels are heuristic, the labelling failure and the evaluation failure are not independent: the first creates the second.

Scope. Two questions are raised here and deliberately left open: which of SZZ’s two error types an online learner cannot survive, and whether a learner can be built to withstand it. Chapter 6 states both designs, together with the specific controls this report has already shown they will require, and records in advance the commitment that the mitigation phase will be pre-registered.

Abbreviations

CI	Confidence interval
FN / FP / TN / TP	False negative / false positive / true negative / true positive
HL	Hodges–Lehmann estimator
JIT-SDP	Just-in-time software defect prediction
MCC	Matthews correlation coefficient
OOB	Oversampling Online Bagging
ORB	Oversampling Rate Boosting
SZZ	Śliwerski–Zimmermann–Zeller algorithm (variants: B-, AG-, MA-, R-, L-, RA-)
p_0 / p_1	Class-conditional false-alarm rate / miss rate
λ	Poisson oversampling rate in online bagging
W	Verification-latency decision window (90 days)

A note on corrections

One finding in this report contradicts a claim made earlier in the same project: a decomposition of the verification-latency effect, withdrawn after the contrast was found not to be identified. The superseded claim is stated together with the design error that produced it, rather than removed (§5.3).

This is deliberate. The correction is part of the evidence — it records which conclusions survived contact with a control and which did not, and the control that overturned it had been built for a different purpose.

Chapter 1 — Introduction

1.1 Context

Defects are expensive in proportion to how long they survive. A fault caught at review costs a developer's attention; the same fault caught in production costs an incident, a rollback and the trust of whoever depended on the system. This asymmetry motivates *defect prediction*: statistical models that direct limited inspection effort toward the code most likely to be wrong.

Classical defect prediction operates at release granularity, predicting which files or modules will contain defects. Its practical difficulty is one of timing. A prediction about a file, delivered at release, arrives long after the change that introduced the problem, and often reaches a developer who did not write it.

Just-in-time software defect prediction (JIT-SDP) moves the unit of prediction from the file to the *commit*. Each change is scored as it arrives, so a risky change can be flagged while its author still has it in mind. Kamei et al. (Kamei et al. 2013) established the modern form of the task, defining a set of change-level metrics — size, diffusion, history, and developer experience — that remain the standard feature set, and reporting roughly 68% accuracy and 64% recall at predicting defect-inducing changes.

The setting is unusual in three ways that jointly shape everything in this report. Commits arrive as a **stream**, so any realistic evaluation is temporal. Defect-introducing commits are **rare** — 8.54% on the corpus studied here — so the task is severely imbalanced. And, critically, **the label for a commit is not available when the commit arrives**. It becomes available only when someone finds the defect, traces it to its origin, and fixes it.

1.2 The ground-truth problem

JIT-SDP requires a label for every commit: did this change introduce a defect? Almost no project records this directly. What projects record is which commits *fixed* defects.

The gap is bridged by the **SZZ algorithm** (Śliwerski, Zimmermann and Zeller), (Śliwerski et al. 2005), which works backwards: take a bug-fixing commit, identify the lines it modified, and use `git blame` to find which earlier commit last touched each of those lines. Those earlier commits are labelled defect-introducing.

The inference is only as good as its premise — that the lines a fix modifies are the lines that were wrong. **That premise fails routinely, because bug-fixing commits are tangled.** Developers reformat while fixing, rename while fixing, refactor while fixing, update comments and tests while fixing. Every such line is blamed by SZZ, and every commit that last touched one is labelled as having introduced a defect it had nothing to do with.

The magnitude is now measured rather than suspected. Herbold et al. (Herbold et al. 2022), using four annotators per line, found that **between 17% and 32% of all changes within bug-fixing commits actually address the underlying problem** — rising to 66–87% when only production code files are considered. The remainder is tangled. Roughly 11% of lines were hard enough to classify that annotators actively disagreed.

A substantial body of work has proposed refinements — AG-SZZ, MA-SZZ, R-SZZ, L-SZZ, RA-SZZ — each adding filters intended to remove the over-collection. Whether those refinements improve the labels, and at what cost, is the subject of Chapter 4.

The consequence for the field is structural. Essentially every JIT-SDP dataset in use descends from SZZ. Models are trained on SZZ labels, evaluated against SZZ labels, and compared with one another on SZZ labels. If those labels are systematically wrong, the entire measurement apparatus is calibrated against a biased instrument — and, because the evaluation uses the same instrument as the training, the bias is invisible from inside.

1.3 The realism problem

A second problem is independent of labels and compounds them.

The standard evaluation protocol in the JIT-SDP literature is random k -fold cross-validation: shuffle the commits, hold out a fold, train on the rest. On temporally ordered data this permits a model to train on 2020 commits and be tested on 2015 ones. The resulting estimate answers a question nobody has: how well would this model do if it could see the future?

Verification latency is the subtler half. Even under a correct chronological split, the assumption that a commit's label is available at training time is false. The label exists only once the defect is found and fixed, and on this corpus the median delay is 113 days, with **53% of defect labels arriving after a 90-day decision window**.

This is worse than a delay. A system with a 90-day window does not simply wait — it concludes that an unfixed commit is clean, trains on that conclusion, and is corrected months later. **A learner under verification latency is actively trained on labels it will subsequently discover were wrong**, and the wrong labels it receives are exactly the defect-introducing commits it most needs to learn from.

Cabral and Minku (Cabral et al. 2019) characterised this setting, showing that the class imbalance itself evolves over time under latency, and proposed **Oversampling Rate Boosting (ORB)** — an online ensemble that adjusts its resampling rate when predictions become biased toward one class. ORB is the online learner used throughout this report. The mechanism it uses to handle imbalance — amplifying minority-class examples in proportion to how rare they appear — is also, on the evidence assembled here, a plausible route by which label noise could do disproportionate damage. Testing that is the next phase of the work and is not attempted here.

1.4 Problem statement

Just-in-time defect prediction is evaluated almost exclusively against labels produced by SZZ, using protocols that ignore both commit order and verification latency. The field therefore has **no measurement of what SZZ-induced label noise costs a model under realistic deployment conditions**, and no way to distinguish a model's ability to find defects from its ability to reproduce the heuristic that labelled them.

Four things must be separated to make progress: the noise in the labels; the inflation contributed by the evaluation protocol; the *mechanism* by which noise damages an online learner; and whether a learner can defend itself.

This report covers the first two. Phase 1 characterises the noise; Phase 2 measures what it costs downstream and separates that cost from the inflation contributed by the evaluation protocol. The remaining two are stated as research questions, their designs are given in Chapter 6, and **no result is claimed for either**.

1.5 Research gap

Three lines of work approach this territory without closing it.

SZZ evaluation research measures how accurately SZZ variants identify defect-introducing commits, including against developer-informed oracles. It establishes that the labels are noisy. It stops at the labels and does not measure the downstream cost.

Online JIT-SDP research addresses class imbalance evolution and verification latency, and Cabral and Minku's ORB includes a safety mechanism against potentially noisy minority-class examples. This work treats noise as a nuisance to be dampened rather than as a measurable quantity with a known, asymmetric structure and a known origin.

Learning with noisy labels provides loss correction, sample selection and confidence-based methods. These are almost entirely batch methods developed for symmetric or uniform label noise, and do not transfer directly to a stream in which labels arrive late, arrive wrong, and are later corrected.

The gap is the intersection. No existing work measures the cost of *SZZ-origin* noise — asymmetric, variant-dependent, with known flip rates — under *latency-aware online* evaluation, and no work isolates which of SZZ’s two error types an online learner actually cannot survive. This report occupies that intersection, and its central methodological device is the one that makes the intersection measurable: **holding out an independently constructed reference label set so that a model trained on SZZ can be scored against something other than SZZ.**

1.6 Research questions

RQ1. How much do SZZ variants disagree with one another and with an independently constructed reference, and what is the direction of each variant’s labelling bias? (*Chapter 4*)

RQ2. How does the choice of SZZ label source shift measured JIT-SDP performance across naive, chronological, and prequential-with-latency evaluation regimes? (*Chapter 5*)

RQ3. Through what mechanism does label noise degrade an online learner — specifically, how does ORB’s oversampling response behave under symmetric versus SZZ-calibrated asymmetric noise as the dose increases? (*Not addressed in this report; design in §6.2*)

RQ4. Can an online learner recover performance lost to SZZ-origin noise without access to clean labels, and without sacrificing performance when the labels are clean? (*Not addressed in this report; design in §6.2*)

1.7 Contributions

A class-conditional characterisation of SZZ noise. Six variants scored over an identical 27,319-commit denominator: a precision ceiling of 27.2%, bifurcated bias (ρ_0 0.067–0.263, ρ_1 0.359–0.733), and inter-variant agreement of κ up to 0.933 against reference agreement of κ 0.158–0.202. **Variants share failure modes, so their mutual agreement is not evidence of correctness.** The flip-rate pairs are released as a calibration artefact.

A separation of protocol inflation from label effects. Random k -fold inflates a high-capacity model by +0.137 MCC (21 of 21 projects); self-scoring inflates it by a further +0.230. The two protocol effects together are roughly nine times the label-source effect, and the inflation scales with model capacity.

A calibrated noise model released as an artefact. The class-conditional flip rates (ρ_0 , ρ_1) for all six variants, estimated over a common denominator and serialised for reuse, so that downstream noise studies can calibrate against measured SZZ behaviour rather than assume symmetric noise.

A reproducible pipeline with an integrity gate. The full study runs on a continuous-integration runner from two small committed files, behind an assertion that recomputes the training data’s confusion matrix and refuses to spend compute if it disagrees with the reported noise rates.

1.8 Thesis organisation

Chapter 2 surveys JIT-SDP, SZZ and its variants, evaluation methodology, online learning under latency, and learning with noisy labels, and positions this work against each.

Chapter 3 states the research design: the phase pipeline, the corpus and its reference labels, the SZZ toolchain, the metrics and statistical protocol, and the validity framework — stated in advance rather than assembled afterwards.

Chapters 4 and 5 report Phases 1 and 2, answering RQ1 and RQ2 respectively.

Chapter 6 concludes: it synthesises the two results, gives the full treatment of validity threats, and sets out the design of the remaining phases together with what this report establishes that they now depend on.

A note on how the results chapters are written. One finding in this report contradicts a claim made earlier in the same project: a decomposition of the verification-latency effect, which was withdrawn after it was found not to be identified. The superseded claim is **stated, together with the design error that produced it**, rather than removed — see §5.3. The correction is part of the evidence: it records which conclusions survived contact with a control and which did not.

Chapter 2 — Background and Related Work

Citation status. Every reference in this chapter was checked against its publisher record or the source itself during preparation; §2.9 records the outcome. **One cited work could not be located and has been removed** — see §2.9. Page-level details (exact page numbers, section numbers) should still be confirmed against the PDFs when the bibliography is typeset.

2.1 Just-in-time software defect prediction

2.1.1 Task definition

JIT-SDP predicts, for each incoming commit, whether it introduces a defect. Formally, given a commit c with feature vector $x \in \mathbb{R}^d$ available at commit time, predict $y \in \{0, 1\}$.

The task’s appeal is timing. A prediction available at commit time reaches the developer who wrote the change while the change is still current, which is the difference between a useful signal and a retrospective one.

2.1.2 The Kamei feature set

Kamei et al. (Kamei et al. 2013) established the modern formulation, together with the change-level metrics that remain standard. The 14 features group into five families:

Family	Features	Intuition
Diffusion	NS, ND, NF, Entropy	A change spread across many subsystems, directories and files is harder to reason about
Size	LA, LD, LT	Large changes carry more opportunity for error
Purpose	FIX	Changes that fix defects are themselves defect-prone
History	NDEV, AGE, NUC	Code touched by many developers, recently, and often
Experience	EXP, REXP, SEXP	Author familiarity with the code being changed

Kamei et al. report approximately **68% accuracy and 64% recall** on defect-inducing changes (Kamei et al. 2013). These features are used throughout this report, taken as published rather than recomputed so that feature extraction cannot differ between the reference labels and the SZZ variants.

2.1.3 Class imbalance and concept drift

Defect-introducing commits are a small minority — **8.54%** on the corpus studied here. Two consequences run through this report.

Accuracy is uninformative. A model predicting “clean” for every commit scores 91.5% accuracy on this corpus. §2.6 covers metric selection.

Oversampling is standard, and it is not free. Rebalancing methods increase the influence of minority examples, which is exactly what is wanted when those examples are correct. What happens when they are *not* correct is the question Phase 3 of this project is designed to answer; it is outside the scope of this report.

Projects also change over time — team composition, architecture, process — so the relationship between features and defect-proneness is not stationary. Concept drift in this setting is addressed by Cabral and Minku (Cabral et al. 2019; Cabral and Minku 2023) and by subsequent work; §2.4 returns to it.

2.2 SZZ and its variants

2.2.1 The original algorithm

Śliwerski, Zimmermann and Zeller, in *When Do Changes Induce Fixes?* (Śliwerski et al. 2005), introduced the procedure now universally called SZZ after their initials. Given a commit identified as fixing a defect, it:

1. Extracts the lines modified by the fix;
2. Applies `git blame` (or equivalent) to find, for each line, the most recent prior commit to have modified it;
3. Labels those prior commits defect-introducing.

The algorithm is an inference, and it rests on two premises: that the lines a fix modifies are the lines that were defective, and that the last commit to touch a line is the commit that made it wrong.

2.2.2 The failure taxonomy

Tangled commits violate the first premise. Bug-fixing commits routinely contain reformatting, renaming, comment updates and test changes alongside the fix. Herbold et al. (Herbold et al. 2022), using four annotators per line, found that **between 17% and 32% of all changes in bug-fixing commits address the underlying problem**, rising to **66–87%** when only production-code files are considered; about **11%** of lines produced active annotator disagreement. Every tangled line is blamed, and every commit that last touched one is mislabelled.

Blame attribution error violates the second premise. `git blame` identifies the last commit to *touch* a line, which need not be the commit that made it *wrong*. A later cosmetic edit takes the blame from the change that introduced the fault. This report refers to this as the **syntactic line-blame fallacy**, and §2.2.4 explains why it is not measurable in this study’s design.

Structural invisibility. A defect introduced by omission — code that was never written — has no line for blame to find. This mechanism is real in general but, as Chapter 4 explains, is *not available* as an explanation within this report’s corpus.

2.2.3 The variants

Each variant adds filtering or selection intended to remove B-SZZ’s over-collection:

Variant	Mechanism	Attribution
B-SZZ	The original	Śliwerski et al. (2005)
AG-SZZ	Annotation graphs for precise line tracing; filters cosmetic changes such as blank lines and comments	Kim et al. (2006)
MA-SZZ	Additionally excludes meta-changes — branch merges, property changes — that do not alter program behaviour	Costa et al. (2017)
R-SZZ	Retains only the most recent candidate commit	Davies et al. (2014)
L-SZZ	Retains only the candidate with the most changed lines	Davies et al. (2014)
RA-SZZ	Refactoring-aware; detects and filters refactorings via RefDiff and RefactoringMiner (Java only)	Neto et al. (2018)

The progression assumes that B-SZZ’s dominant error is over-collection and that filtering it out improves the labels. **Chapter 4 tests that assumption directly and finds that for the most aggressive variants, more than half of what the filters remove is correct.**

2.2.4 Developer-informed oracles

Evaluating SZZ requires a reference that does not itself use SZZ. Rosa et al. (Rosa et al. 2021, 2023) constructed one by mining commit messages in which developers *explicitly name* the commit that introduced the defect being fixed — yielding **1,930 fix-to-inducing links across eight programming languages**, extended in the journal version to an evaluation of nine variants over **2,304 instances**. Against that oracle, **R-SZZ performed best (F-measure $\approx$ 61%, precision $\approx$ 66%)** while B-SZZ achieved higher recall ($\approx$ 0.69) at markedly lower precision ($\approx$ 0.39).

Two points about this dataset matter for the present work.

First, it **cannot train a JIT-SDP model**. It contains fix-to-inducing links and no clean commits, so there is no negative class and no complete labelling of project history.

Second, **its precision figures are not comparable to this report’s without reconciling the denominator**. Rosa et al. score *conditionally* on bug-fixing commits already known to have an inducing commit. This report scores over an entire project history of 27,319 commits, the overwhelming majority of which are clean. A variant that is right two times in three when asked “which earlier commit introduced this known defect?” can still be wrong four times in five when asked “is this commit, drawn from all of history, defect-introducing?” **The difference is large enough to read as a contradiction, and Chapter 6 addresses it explicitly.**

2.2.5 JIT-Defects4J and the reference labels

Ni et al. (Ni et al. 2022) published **JIT-Defects4J**, an extension of LLTC4J (Herbold et al. 2022), comprising **27,319 commits from 21 Java projects, of which 2,332 (8.54%) are labelled defect-introducing**. Its construction is two-stage: lines within bug-fixing commits were annotated manually, retained where at least three participants agreed; `git blame` was then applied to those verified lines.

The labels are therefore `git blame` seeded with human-verified fix lines — **not manually verified ground truth for defect-introducing commits**. This report uses them as its reference precisely because the distinction is tractable: they control for tangling, the dominant false-positive mechanism, while sharing SZZ’s blame step. Comparisons against them isolate the cost of tangled commits, and blame error, being common to both sides, cancels. Chapter 3 §3.2.2 develops the consequences.

2.2.6 Other JIT-SDP datasets

Dataset	Scale	Labels	Relevance here
ApacheJIT (Keshavarz and Nagappan 2022)	106,674 commits (28,239 bug-inducing), 14 Apache projects	SZZ-derived	Larger, but cannot serve as an oracle for a study of SZZ noise
Defectors (Mahbub et al. 2023)	~213K files, 24 Python projects, 18 domains	SZZ-derived	File-level; same objection
ReDef	3,164 defective / 10,268 clean functions, 22 C/C++ projects	Revert-anchored , LLM-triaged, ~92% audited precision	Non-blame labels, but function-level, C/C++, no Kamei features, no fix dates

Scale is not the binding constraint for this report; label provenance is. A study measuring what SZZ noise costs cannot use SZZ labels as its reference, which excludes the two largest datasets available.

2.3 Evaluation methodology in defect prediction

2.3.1 Temporal leakage

Random *k*-fold cross-validation on temporally ordered data allows a model to train on future commits and be tested on past ones. Time-aware evaluation — training only on the past and evaluating only on the future — has been advocated repeatedly, notably by Falessi and colleagues (Falessi et al. 2020), who caution researchers against “time travel” and against generalising beyond the evaluation context. The reported effect is consistent in direction with this report’s: in-sample performance is systematically inflated relative to time-aware performance. Random *k*-fold nonetheless remains

widespread in the JIT-SDP literature: the systematic survey of the field (Zhao et al. 2023) catalogues 67 studies, and a recent audit of 101 defect-prediction papers (Destefanis et al. 2026) assessed design, analysis and reporting practice against accepted norms and found both wanting.

Chapter 5 measures the consequence rather than restating the argument. The inflation is +0.137 MCC on a high-capacity model and it scales with model capacity, which means comparisons *between* model families under this protocol are confounded in a direction that favours larger models.

2.3.2 Prequential evaluation

For streams, the standard protocol is *test-then-train*: each example is first predicted, then used for learning. Summarising a metric over a stream requires a forgetting mechanism, conventionally a fading factor applied to the error estimate. Gama, Sebastião and Rodrigues (Gama et al. 2013) defend prequential error with forgetting and prove that, **for consistent learning algorithms on stationary data, the prequential error computed with fading factors converges to the Bayes error.**

An important caveat for this report. That convergence result is established for an *error rate* — a decomposable loss accumulated example by example. **MCC is not decomposable**, being a non-linear function of all four confusion-matrix cells, so neither the construction nor its convergence guarantee extends to it directly. This report therefore reports two ad-hoc summaries of the MCC trajectory — a terminal fading value and a trajectory mean — treats the trajectory mean as primary **on measured variance rather than by appeal to a standard**, and states explicitly that neither is canonical. An earlier version of this work described one of them as “the standard estimator”; that was incorrect and is corrected in Chapter 3 §3.4.2.

2.3.3 Verification latency

Cabral and Minku (Cabral et al. 2019) identified verification latency as a first-class concern in JIT-SDP: labels arrive with a delay, and the delay differs systematically between classes, since a clean commit is only ever *presumed* clean while a defective one must be discovered. Their framing of latency and of the resulting **class imbalance evolution** is the setting this report adopts.

On the corpus studied here the median latency is **113 days**, with **53% of defect labels arriving after a 90-day window**. Under a fixed window, a defect label that arrives late is delivered first as a *wrong clean label* and corrected later — so latency is not merely delay but active misinformation, and the misinformation concerns exactly the minority class.

2.4 Online learning for JIT-SDP

Online Bagging (Oza and Russell 2001) adapts bagging to streams by presenting each arriving example to each ensemble member $k \sim \text{Poisson}(\lambda)$ times, achieving performance comparable to batch bagging without storing the dataset.

Oversampling Online Bagging (OOB) (Wang et al. 2015) handles imbalance by setting λ from the running observed class ratio — tracked with time-decayed metrics — so minority examples are presented more often as the imbalance grows. OOB and its undersampling counterpart UOB are the standard data-level approaches for class-imbalanced streams.

Oversampling Rate Boosting (ORB) extends OOB by monitoring the model’s own recent predictions: when predictions are considerably biased toward one class, the resampling rate of the opposite class is boosted. Cabral and Minku also adopt a safety mechanism intended to avoid training on potentially noisy or outlier minority-class

examples.

This machinery motivates the next phase of the work. ORB's λ is a function of the *observed rate* of defect labels — that is, of label **quantity**. It has no access to label **quality**, so a false positive receives the same amplification as a true one. Whether that amplification is the route by which SZZ noise damages an online learner is a hypothesis this report raises and does not test; Phase 3 is designed to measure the coupling directly.

Cabral and Minku's later work (Cabral and Minku 2023) analyses concept drift in JIT-SDP across $p(y)$, $p(x|y)$ and $p(x)$, reports **defect discovery delays ranging from one day to more than eleven years**, and proposes a drift-adaptive method that monitors predictions on unlabelled data rather than waiting for labels. **Their finding that medians are typically at or below 90 days, and that a 90-day waiting period trades off correct labelling against drift, is the basis for this report's choice of $W = 90$ days. The distinction from the present work is the origin of the noise.** Latency-induced noise is a timing artefact: the label is correct but late. SZZ-induced noise is a *labelling* artefact: the label is wrong and will never be corrected. This report addresses the second, under conditions that include the first.

2.5 The weak-baseline problem

Deep learning approaches to JIT-SDP — **DeepJIT** (Hoang et al. 2019), which encodes commits hierarchically from line to hunk to file to commit using a convolutional text model, and **CC2Vec** (Hoang et al. 2020), which learns code-change representations supervised by commit logs via a hierarchical attention network — reported substantial gains over feature-based models. Two subsequent results complicate that picture.

Zeng et al. (Zeng et al. 2021), evaluating on over **310,370 changes**, found that CC2Vec could not consistently outperform DeepJIT and that neither consistently outperformed traditional feature-based JIT prediction. They then constructed **LApredict** — logistic regression on the added-lines feature alone — which outperformed both deep models while being roughly **81,000–120,000× faster** to train and test.

Pornprasit and Tantithamthavorn (Pornprasit and Tantithamthavorn 2021) reported **JITLine**, a simpler feature-based approach that was **26–38% better in F-measure, 17–51% more cost-effective, and 70–100× faster** than CC2Vec and DeepJIT.

Both models are used in this report, and the reason is methodological rather than deferential. LApredict has minimal capacity; JITLine has a great deal. Chapter 5 shows that the inflation attributable to evaluation protocol is **+0.137 MCC for JITLine and +0.034 for LApredict**, and that the self-scoring gap is significant for all six SZZ variants under JITLine but only one under LApredict. **Capacity is not incidental to the measurement problem — it is the variable that determines its size**, which offers an alternative reading of why simple baselines have proven so hard to beat.

2.6 Metric selection

Under an 8.54% positive rate, accuracy is uninformative. This report uses:

Matthews Correlation Coefficient (MCC) — introduced by Matthews (Matthews 1975) for protein secondary-structure prediction and since adopted widely for imbalanced binary classification — as primary. It incorporates all four confusion-matrix cells and is near zero for any uninformed strategy. A majority-class predictor scores 0.0 MCC and 91.5% accuracy on this corpus.

G-mean, the geometric mean of per-class recalls, which collapses toward zero if either class is abandoned.

Cohen’s κ (Cohen 1960), the proportion of agreement corrected for chance, for agreement between labelling procedures, where the question is concordance rather than prediction quality.

Effort-aware metrics — recall at a fixed inspection budget — arguably match the practitioner’s objective more closely and are identified as future work in Chapter 6, not reported here.

2.7 Learning with noisy labels

Loss correction. Natarajan et al. (Natarajan et al. 2013) study binary classification under random classification noise whose **flip probability is class-conditional** — precisely the noise model Chapter 4 measures — and construct a simple **unbiased estimator of any loss**, with performance bounds for empirical risk minimisation on noisily labelled i.i.d. data. The correction becomes numerically unstable as $\rho_0 + \rho_1 \rightarrow 1$. **This matters directly here:** Chapter 4 measures $\rho_0 + \rho_1$ approaching 0.8 for the conservative variants, which is why the loss-correction arm evaluated in this report is capped.

Sample selection. Co-teaching (Han et al. 2018) trains two networks simultaneously; on each mini-batch, each selects the examples it considers clean and back-propagates only the examples selected by its peer, on the premise that mislabelled examples exhibit higher loss early in training. It assumes a batch setting with multiple epochs and two models.

Confident Learning (Northcutt et al. 2021) assumes a **class-conditional** noise process and directly estimates the joint distribution between noisy and true labels, then prunes, counts with probabilistic thresholds, and ranks examples by confidence. It is a batch method requiring cross-validated out-of-sample predictions over a fixed dataset.

Why these do not transfer directly. Each assumes a fixed dataset, repeated passes, and labels that are wrong but *stable*. A JIT-SDP stream offers none of these: a single pass, labels arriving late, and labels that are **wrong and then corrected**. Adapting the confidence-based family to a stream is the subject of Phase 4 and is not attempted in this report.

2.8 Positioning

Work	Imbalance	Drift	Latency	SZZ noise measured	Online
Kamei et al.	○	○	○	○	○
Cabral & Minku (ORB; drift work)	●	●	●	○	●
Rosa et al. (developer-informed oracle)	○	○	○	● (labels only)	○
Herbold et al. (tangling)	○	○	○	● (labels only)	○
Zeng et al. / Pornprasit & Tantithamthavorn	●	○	○	○	○
Natarajan / Co-teaching / Confident Learning	●	○	○	◐ (noise model assumed, not measured)	○
This report (Phases 1–2)	●	●	●	● downstream cost	●
<i>Planned (Phases 3–4)</i>	●	●	●	◐ <i>mechanism and mitigation</i>	●

(● addressed · ◐ partially · ○ not addressed)

The distinguishing cell is the fourth. SZZ evaluation work measures the noise and stops at the labels. Online JIT-SDP work operates under latency but treats label noise as a nuisance to dampen rather than a quantity to measure. Noisy-label learning assumes a noise model rather than estimating one, and assumes a batch setting. This report measures **what SZZ-origin noise costs downstream, under latency-aware online evaluation**. Whether that cost can be recovered is the subject of the phases that follow it, and no claim about recovery is made here.

The methodological device that makes this possible is small and worth naming: **an independently constructed reference label set, held out from training and used only for scoring**. Without it, a model trained on SZZ can only be evaluated against SZZ, and the largest effect in this report — the +0.230 MCC self-scoring gap — is invisible by construction.

2.9 Citation verification status

Every work cited in this chapter was checked against its publisher record or the source during preparation. The table records what was confirmed.

Work	Venue	Confirmed
Śliwerski, Zimmermann & Zeller, <i>When Do Changes Induce Fixes?</i>	MSR 2005	Original SZZ; fix-inducing change analysis over CVS archives
Kim et al.	2006	AG-SZZ: annotation graphs, cosmetic-change filtering
da Costa et al.	2017	MA-SZZ: excludes meta-changes (branch merges, property changes)
Davies et al.	2014	R-SZZ (most recent candidate), L-SZZ (most changed lines)
Neto et al.	2018	RA-SZZ: refactoring-aware via RefDiff / RefactoringMiner, Java only
Rosa et al.	ICSE 2021; JSS 202:111729, 2023	1,930 links, 8 languages, 2,304 instances; R-SZZ best ($F \approx 61\%$, $P \approx 66\%$); B-SZZ recall ≈ 0.69 , precision ≈ 0.39 ; PySZZ
Herbold et al.	EMSE, 2022	17–32% of changes / 66–87% production code; four annotators per line; ~11% disagreement
Ni et al.	ESEC/FSE 2022	JIT-Defects4J; 27,319 commits / 2,332 defective / 21 projects; two-stage construction
Kamei et al.	IEEE TSE 39(6):757–773, 2013	Task formulation, change-level metrics; ~68% accuracy, 64% recall
Cabral & Minku	ICSE 2019	Class imbalance evolution, verification latency, ORB, safety mechanism
Cabral & Minku, <i>Towards Reliable Online JIT-SDP</i>	—	Drift across $p(y)$, $p(x y)$, $p(x)$; delays from 1 day to >11 years; medians at or below 90 days; 90-day waiting time as the trade-off
Oza & Russell, <i>Online Bagging and Boosting</i>	AISTATS 2001, PMLR R3:229–236	Poisson(λ) online bagging matching batch performance
Wang, Minku & Yao	—	OOB / UOB; time-decayed class-imbalance tracking
Gama, Sebastião & Rodrigues, <i>On Evaluating Stream Learning Algorithms</i>	Machine Learning, 2013	Prequential error with fading factors converges to Bayes error for consistent learners on stationary data
Hoang et al., <i>DeepJIT</i>	MSR 2019	Hierarchical line $\rightarrow$ hunk $\rightarrow$ file $\rightarrow$ commit encoding, textCNN
Hoang et al., <i>CC2Vec</i>	ICSE 2020	Code-change representations supervised by commit logs, hierarchical attention
Zeng et al.	ISSTA 2021	310,370 changes; LAPredict; 81k–120k $\times$ speedup
Pornprasit & Tantithamthavorn, <i>JITLine</i>	MSR 2021, pp. 369–379	26–38% F-measure, 17–51% cost-effectiveness, 70–100 $\times$ faster

Work	Venue	Confirmed
Keshavarz & Nagappan, <i>ApacheJIT</i>	MSR 2022	106,674 commits, 28,239 bug-inducing, 14 projects
Mahbub, Shuvo & Rahman, <i>Defectors</i>	MSR 2023	~213K files, 24 Python projects, 18 domains
ReDef	preprint, 2025	3,164 / 10,268 function-level changes, 22 C/C++ projects, revert-anchored, ~92% precision
Matthews	BBA 405(2):442–451, 1975	MCC, originally for protein secondary-structure prediction
Cohen	<i>Educ. Psychol. Meas.</i> 20:37–46, 1960	κ as agreement corrected for chance
Natarajan et al., <i>Learning with Noisy Labels</i>	NIPS 2013	Class-conditional flip probability; unbiased loss estimator; ERM bounds
Han et al., <i>Co-teaching</i>	NeurIPS 2018	Two networks exchanging small-loss examples per mini-batch
Northcutt, Jiang & Chuang, <i>Confident Learning</i>	JAIR 70, 2021	Class-conditional noise; joint distribution estimation; prune, count, rank
Zhao, Damevski & Chen	ACM CSUR 55(10):1–35, 2023	Systematic survey of JIT-SDP; 67 studies
Destefanis et al., <i>An Audit of ML Experiments on Software Defect Prediction</i>	EMSE, 2026	101 audited papers; SCOPUS 2019–2023; design, analysis and reporting practice
Falessi et al.	—	Time-aware evaluation; caution against “time travel”; in-sample performance systematically inflated

Two notes on the bibliography

One cited work was removed. The project’s planning documents referenced “Song et al. (2022)” as a method addressing latency-induced label noise in JIT-SDP. **That work could not be located.** Searching the area returns Cabral and Minku on verification latency and Tabassum, Minku and colleagues on waiting-time labelling, either of which may have been what was intended. The reference is therefore omitted rather than guessed at, and §2.4’s claim about latency-noise methods is attributed only to work that was confirmed. **If a specific Song et al. paper was intended, locate it before reinstating the citation.**

One attribution needs confirming. A deep-learning-in-SDP systematic review and meta-analysis exists (*Information and Software Technology*, 2023) and is the work the planning documents meant by “Zain et al.”, but the first-author attribution was not confirmed. It is not cited in this chapter; if it is added, check the author list.

One correction to the project’s own outline. An earlier outline attributed to Herbold et al. a finding that “~50%” of SZZ labels are wrong. The verified figures are **17–32% of all changes in bug-fixing commits** addressing the underlying problem, rising to **66–87%** for production-code files only — a different quantity, measured at line rather than commit level. The “~50%” figure does not appear in the source and must not be cited.

Remaining work for the bibliography. Page numbers and DOIs should be taken from the publisher records when the reference list is typeset; a few entries above carry venue and year but not pagination. Nothing in the chapter’s prose depends on those details.

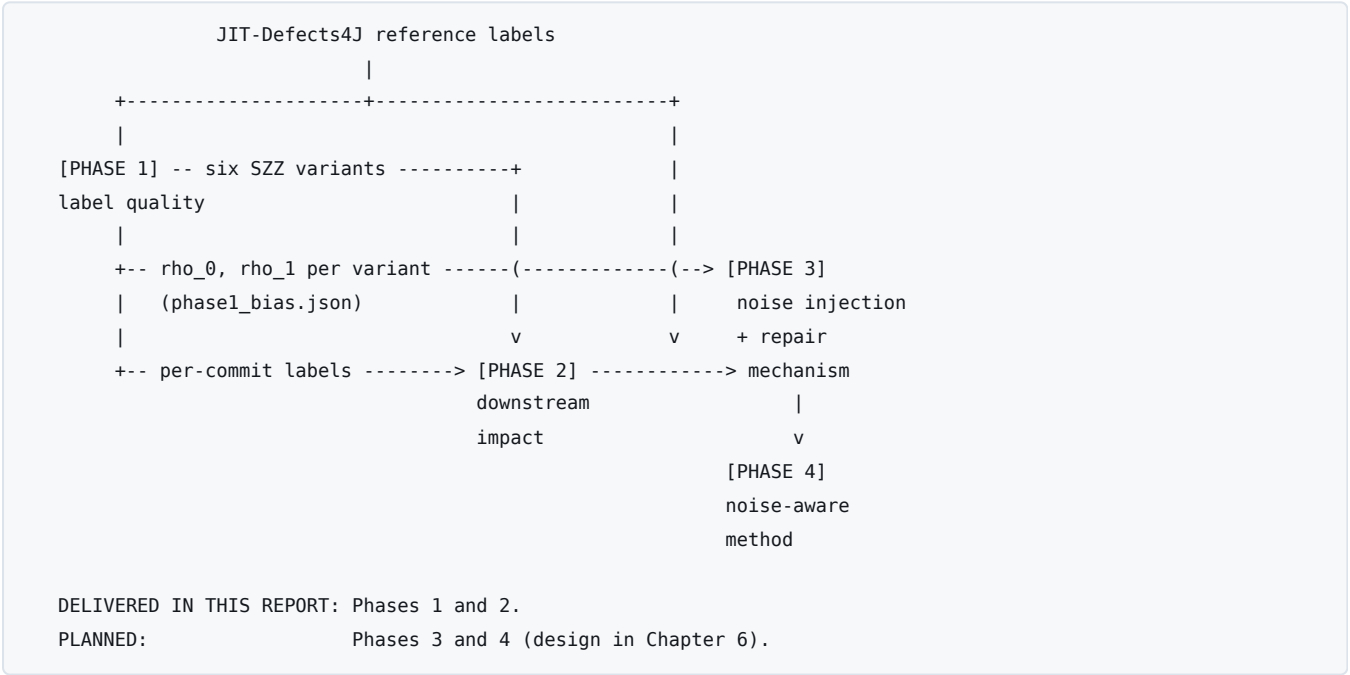
Chapter 3 — Research Design

This chapter states the design of the study as a whole: how its phases fit together, which two of them this report delivers, what data they run on, how the SZZ variants were produced, what is measured and with what statistical protocol, and which validity threats were anticipated in advance rather than discovered afterwards.

The design has one organising commitment. **Every claim is a paired comparison at the project level, and every reported number is regenerable from committed inputs by a named script.** The remainder of this chapter is largely an elaboration of what that commitment required.

3.1 The phase pipeline

Each phase consumes the previous phase’s output, and the dependencies are data dependencies rather than narrative ones.



- **Phase 1** measures how far each SZZ variant departs from the reference, and in which direction. Output: per-commit labels, and the flip-rate pair (ρ_0, ρ_1) per variant.
- **Phase 2** trains models on each label source and evaluates them under four regimes of increasing realism, against two scoring conventions. Output: the inflation ladder and the label-source penalty.
- **Phase 3** (*planned*) will use Phase 1’s flip rates to inject calibrated noise into clean labels, and separately repair real SZZ labels one error type at a time, to identify which error type costs the learner and by what mechanism.
- **Phase 4** (*planned*) will build and pre-register a mitigation targeting whatever Phase 3 identifies.

The dependencies are data dependencies, and they run one way. Phase 3 cannot be calibrated without Phase 1’s flip rates, and Phase 4’s design is determined by whatever Phase 3 finds. That ordering is why the later phases are not pre-specified in detail here: committing now to a mitigation for an error type Phase 3 has not yet identified would be designing the answer before the question.

3.2 Data

3.2.1 Corpus

Projects	21 Apache Java repositories
Commits	27,319
Reference defect-introducing	2,332 (8.54%)
Range per project	544–4,026 commits; 19–335 defects; 1.8%–19.3% defect rate
Features	The 14 Kamei change-level metrics

Projects were not selected by us. The corpus is JIT-Defects4J as published, adopted whole so that the selection cannot be tuned to the results. Its consequences for external validity are stated in §3.5 and Chapter 6.

3.2.2 The reference labels

`label_oracle` derives from JIT-Defects4J (Ni et al. 2022), an extension of LLTC4J (Herbold et al. 2022). Its provenance was traced through the repository and confirmed against the source paper rather than assumed:

Step	Evidence
<code>label_oracle</code>	= <code>is_buggy_commit</code> in the published feature pickles
The pickles	from the JIT-Fine replication package on Zenodo
Corpus check	the paper’s own table reports $2,332 / 27,319 = 8.54\%$ and matches this corpus project by project

Construction, stated precisely because every downstream claim depends on it. Lines within bug-fixing commits were annotated manually, retained only where at least three participants agreed; `git blame` was then applied to those verified lines to identify the introducing commit; everything else is clean by residual.

The reference is therefore **`git blame` seeded with human-verified fix lines**, not manually verified ground truth for defect-introducing commits. The phrase “developer-verified ground truth” is inaccurate and is not used in this report. Chapter 4 §4.1.1 develops the three consequences; the most important is that all measured noise is a **lower bound**, because blame error is present on both sides of every comparison and cancels.

3.2.3 Features and preprocessing

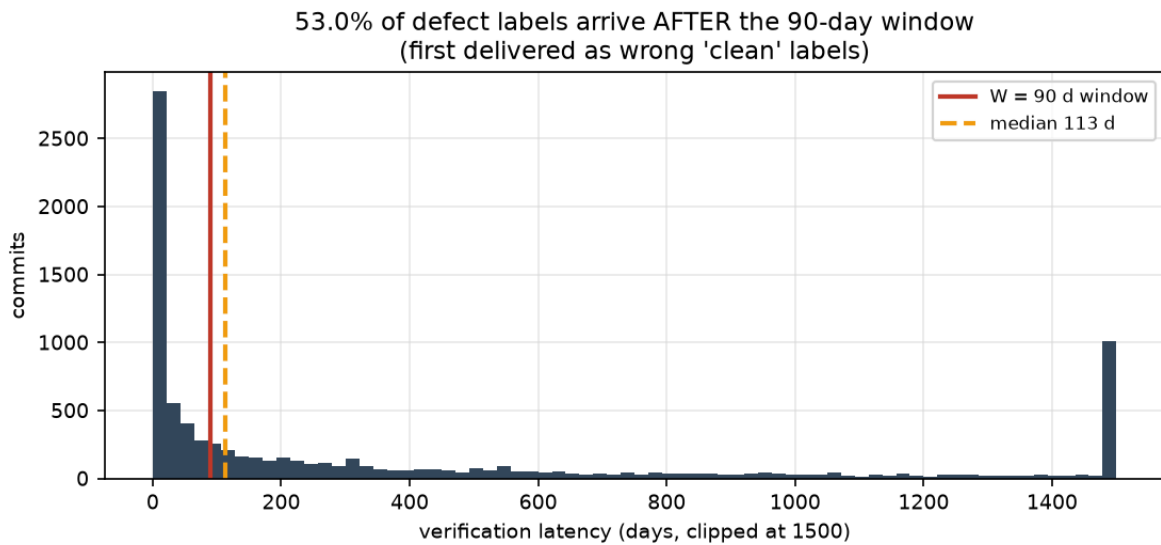
The 14 Kamei metrics are taken as published rather than recomputed, so that feature extraction cannot differ between the reference and the variants. Commits are deduplicated on (project, commit hash) and sorted by author timestamp, which defines the stream order for every temporal regime.

The pipeline was restructured so that two bulky inputs — a 101 MB archive and 830 MB of cloned repositories — are replaced by two small committed derived files (4.2 MB and 0.37 MB). The entire study therefore runs on a continuous-integration runner from committed inputs, which is what makes the reproducibility claim in §3.6 checkable rather than aspirational.

3.2.4 Verification latency

Reconstructing when a defect label would actually have become known requires the author date of each bug-fixing commit. These were extracted once from the cloned repositories and committed as a small table. For each variant, a commit’s arrival time is the date of the fix that blamed it; the union across variants supplies the reference condition’s

arrival times.



Distribution of verification latency against the 90-day decision window

Figure 3.1 — Reading the figure. How long after a commit its defect label actually becomes available. The red line marks the 90-day decision window; the orange dashed line the median, at 113 days. **Over half the distribution lies to the right of the red line.** Those labels arrive too late to be used inside the window, and are first delivered to the learner as *wrong clean labels*.

This is the study’s central construct-validity threat and is declared here rather than in a footnote. The published reference labels carry no native fix-to-inducing linkage, so the reference condition is blame-derived in *construction* and **SZZ-derived in timing**. A coverage sensitivity analysis (67.8% → 100% linkage) bounds part of the exposure; it does not address the timing itself. Every streaming result in Chapters 5–7 inherits this threat.

3.3 SZZ variant implementations

Six variants were produced with **PySZZ v2** (Rosa et al. 2023), the reference implementation released with the developer-informed-oracle study. Using the authors’ own implementation rather than a reimplementer removes one source of unexplained variation, at the cost of inheriting whatever its defaults encode — a trade recorded here as a decision rather than left implicit.

Each variant was run on the same fix-commit set, and its output aligned to the reference over an **identical 27,319-commit denominator**. Partial denominators are a known source of incomparability: a variant emitting fewer candidates can appear more precise merely by being scored on a smaller population.

3.3.1 The label-consistency gate

Mid-project it emerged that the Phase 2 dataset cache had gone stale relative to a Phase 1 label regeneration. Models had been trained on labels one commit older than the noise rates being reported, affecting 1.0–6.5% of commits per variant. The root cause was an unguarded cache keyed on modification time.

The remediation was three-part: a **content-hash staleness guard**, a **provenance sidecar** recording the SHA-256 of every label file used to build the dataset, and an automated **consistency gate** that recomputes the confusion matrix from the training data and asserts it matches the reported noise rates for all six variants. The gate runs in CI **before any**

compute is spent, so a mismatch costs two minutes rather than several hours. It was verified in both directions: it fails on the stale data and passes on the corrected data.

This is recorded in the thesis rather than quietly fixed. A study about label-provenance fragility encountered exactly that fragility in its own pipeline, and now ships an executable assertion against its recurrence.

3.4 Metrics and statistical protocol

3.4.1 Metric selection

MCC is primary. At an 8.54% positive rate, accuracy is uninformative: a model predicting “clean” for every commit scores 91.5% accuracy and 0.0 MCC. MCC uses all four confusion-matrix cells and is near zero for any uninformed strategy, which makes it the appropriate primary under this imbalance. **G-mean** is reported alongside as the geometric mean of per-class recalls, since it is sensitive to abandoning the minority class in a way MCC partially shares but expresses differently. **Cohen’s κ** is used for agreement between labelling procedures, where the question is concordance rather than prediction.

3.4.2 The prequential estimator

Under streaming evaluation the metric is a trajectory, not a scalar, and summarising it requires a choice. Two summaries are computed throughout:

- **Terminal fading** — a confusion matrix with exponential forgetting (fading 0.99, effective window ≈ 100 commits), read at end of stream.
- **Time-averaged** — the mean of the MCC trajectory over the stream.

Neither is canonical, and this report does not claim otherwise. MCC is not a decomposable loss, so the prequential-with-fading construction does not extend to it directly; of the two, the terminal value is the closer analogue. The time-averaged value is treated as **primary on a measured basis** — roughly half the project-level variance (0.051 against 0.092) — not by appeal to a standard. An earlier version of this work described it as “the standard estimator”, which was wrong and is corrected.

Both are reported for every streaming result. They agree on every comparison in this report except one, which is named where it occurs.

3.4.3 The unit of analysis

Every test is paired at the project level, $n = 21$, with seeds averaged before pairing. Seeds control model initialisation and fold randomness; they are not independent observations. Pooling over the full record count would be pseudo-replication, and is the first objection a methodologist would raise.

3.4.4 Effect sizes

Each comparison reports:

- **Wilcoxon signed-rank p** (Wilcoxon 1945) — paired, distribution-free.
- **Matched-pairs rank-biserial correlation** — derived from the same signed ranks as the p -value, so the effect size and the test agree by construction. It reads as *consistency*: +1.000 means every project moved in the same direction.
- **Hodges–Lehmann estimator** (Hodges and Lehmann 1963) — the median of pairwise Walsh averages, the location estimate that accompanies the signed-rank test.

- **Percentile bootstrap confidence interval**, computed **for the Hodges–Lehmann statistic itself**, with the statistic recomputed inside every resample.

Two of these are corrections to an earlier protocol and are recorded as such. Cliff’s δ was initially computed *unpaired*, all-versus-all with denominator $n \cdot m$, discarding the pairing the design exists to preserve; on the headline comparison it reported 0.74 where the paired statistic reports +1.000. Separately, the bootstrap originally resampled the median while the Hodges–Lehmann estimate was reported beside it — an interval for a different quantity than the point estimate.

3.4.5 Multiplicity

Seventy-four tests are run in Phase 2 alone. Multiplicity is corrected **twice**:

- **Holm within test family** (Holm 1979) — families grouped by the comparison being made.
- **Holm globally across all tests** — no family argument required.

A Benjamini–Hochberg false-discovery-rate correction (Benjamini and Hochberg 1995) is computed alongside both and is carried in the released result tables, but no claim in this report rests on it: where Holm and BH disagree, the Holm-corrected value is the one reported.

The global column exists because the families were defined *during* analysis, not declared in advance. A sceptic can reasonably argue the grouping was chosen to help the results; **a claim surviving global Holm needs no defence of how families were drawn**. Every headline claim in this report survives the global column, and where a claim survives only within-family correction, that is stated.

3.4.6 Pre-registration

Phases 1–3 are exploratory. Their hypotheses were formed during analysis and every result is labelled accordingly; the global multiplicity correction is the sensitivity analysis that makes them defensible without a pre-registration.

Nothing in this report is pre-registered. Phases 1 and 2 are exploratory: their hypotheses were formed during analysis and their test families were defined after seeing the data. This is stated wherever a result is reported, and the **global** multiplicity correction is the sensitivity analysis that makes the headline claims defensible without a registration — a claim surviving global Holm needs no argument about how families were drawn.

The mitigation phase will be pre-registered, with hypotheses, primary model, acceptance bar and correction committed to version control before its first run. That commitment is made here, in advance, and §6.2 records it as part of the design.

3.5 Threats to validity, framed in advance

Stating the framework before the results appear is deliberate: it prevents the threat list from being assembled to fit whatever was found.

Construct validity. The reference is `git blame` seeded with verified fix lines, not ground truth, so comparisons isolate tangling rather than total labelling error and measured noise is a lower bound. Arrival times for the reference condition are SZZ-derived (§3.2.4) — the single most serious threat in the study.

Internal validity. Temporal leakage is the threat the design exists to expose, and is therefore a *manipulated variable* rather than a defect. Cache staleness is addressed by the gate (§3.3.1). Repair and injection experiments are oracle-assisted by construction and are labelled as upper bounds, not methods.

External validity. One benchmark family: 21 Apache Java repositories of a single dataset lineage. The corpus is SZZ-shaped, since changes adding no new lines were excluded by the dataset authors, so defects introduced purely by deletion cannot appear. Projects are weighted equally despite an eighteen-fold range in defect count.

Conclusion validity. With 21 paired projects, effects below roughly 0.04 MCC are not resolvable — a limit that becomes load-bearing for the latency decomposition, which is reported as unresolved rather than as null. Multiplicity is corrected twice. The number of exploratory comparisons is large, which is precisely why the global correction is reported.

One scope limit declared in advance. Because the reference labels are themselves blame output, every reference positive is blame-reachable by construction. **No false negative measured against this reference can be attributed to blame’s structural inability to trace a defect.** Any later phase can therefore study false-negative noise arising from over-filtering and seed-line divergence — the two mechanisms Chapter 4 shows are actually operating — but can make no claim about blame-unreachability. Declaring this in advance prevents an unavailable explanation from being offered afterwards.

3.6 Reproducibility

Every number in Chapters 4 and 5 is regenerable from committed inputs by a named script, and the chapter-to-artefact mapping is explicit:

Chapter	Script	Principal outputs
4	<code>experiments/evaluate_confusion_matrix.py</code>	quality table, κ matrix, <code>phase1_bias.json</code>
5	<code>experiments/run_phase2_impact.py</code>	regime $\times$ label-source matrix, 74 tests
Appendix	<code>experiments/run_robustness_checks.py</code>	project table, floor sensitivity, mechanism test

Runtime dependencies are pinned to the exact versions every result was computed with, so a runner reproduces the local environment rather than approximating it. The label-consistency gate runs before every experiment in CI.

The next two chapters report what these scripts produce.

Chapter 4 — Phase 1: Label Disagreement and Oracle Comparison

This chapter answers **RQ1**: how much do SZZ variants disagree with one another and with an independently constructed reference, and what is the direction of each variant’s labelling bias?

The chapter is deliberately placed before any predictive modelling. Chapter 5 is conditioned on the noise characterised here: the reference labels defined in §4.1 are the scoring key used throughout it, and the flip rates estimated in §4.3 are the calibration artefact the later phases will consume.

4.1 Method

4.1.1 The reference condition, and what it is not

Every comparison in this report is made against `label_oracle`, taken from JIT-Defects4J (Ni et al. 2022), itself an extension of LLTC4J (Herbold et al. 2022). Because the interpretation of every subsequent result depends on exactly what these labels are, we state their construction precisely.

The dataset authors describe a two-stage procedure. First, within each bug-fixing commit, individual *lines* were annotated as contributing to the fix, with a line retained only where **at least three participants assigned it the same label**. Second, for each such verified fix line, **git blame was used to identify the commit that introduced it**. Commits not flagged by this procedure are treated as clean by residual.

Three consequences follow, and they constrain the claims this report is entitled to make.

The reference is not manually verified ground truth for defect-introducing commits. Human verification applies to the *fix lines*. The step from a fix line to an introducing commit is algorithmic. The correct description — used consistently throughout this report — is **git blame seeded with human-verified fix lines**. The phrase “developer-verified ground truth” is inaccurate and is not used.

The comparison therefore isolates tangling, not total labelling error. SZZ blames *every* line modified in a bug-fixing commit, including refactoring, formatting and comment changes, then applies filters to remove what it over-collected. The reference blames *only* the lines three annotators agreed were fixing the bug. Both then apply the same blame step. What separates them is exactly one thing: knowledge of which lines in a fix actually fix the bug. That is the definition of a tangled commit, and it is what these comparisons measure.

All measured noise is consequently a lower bound. Any systematic error in `git blame` — the syntactic line-blame fallacy by which blame attributes a line to the last commit that touched it rather than the one that made it wrong — is present on *both* sides of every comparison and cancels. Noise measured against genuine ground truth would be larger by an unknown amount.

This is a weaker claim than “we compare SZZ against truth”, and a more precise one. It is also the claim the data actually support.

4.1.2 Variants, corpus and alignment

Six SZZ variants were executed using PySZZ v2 (Rosa et al. 2023), the reference implementation released with the developer-informed-oracle study:

Variant	Refinement over its predecessor
B-SZZ	The original: blame every line modified in the fix
AG-SZZ	Annotation-graph filtering; excludes cosmetic and format-only changes
MA-SZZ	Additionally excludes meta-changes such as branch and property changes
R-SZZ	Of the candidate inducing commits, retains only the most recent
L-SZZ	Retains only the largest candidate
RA-SZZ	Refactoring-aware: removes candidates attributable to refactorings

The corpus is 21 Apache Java projects, 27,319 commits, of which 2,332 (**8.54%**) carry a reference defect label. Each variant’s output was aligned to the reference over an **identical 27,319-commit denominator**, so that precision and recall are computed against the same population in every row. This matters: partial denominators are a common source of incomparability in the SZZ literature, since a variant that emits fewer candidates can appear more precise merely by being evaluated on a smaller set.

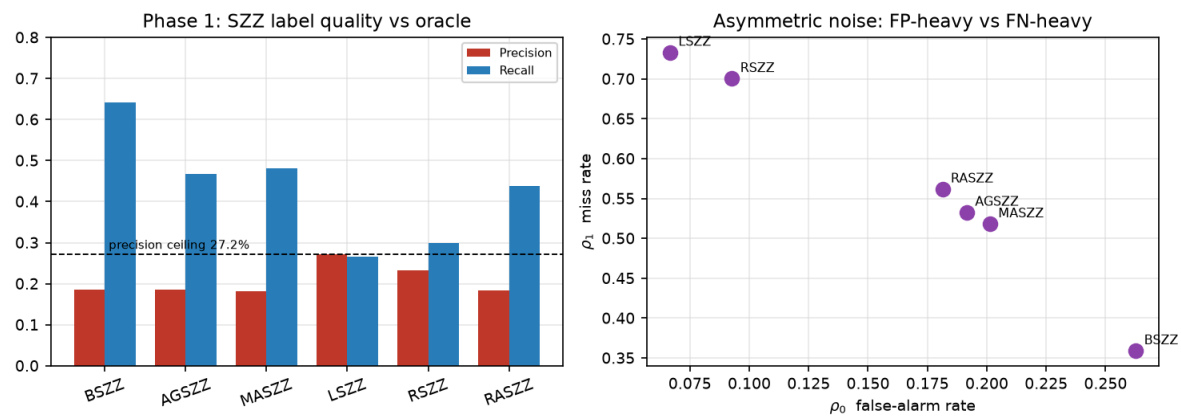
Per-variant quality is reported as precision, recall, F1, MCC and Cohen’s κ , together with the two flip rates defined in §4.3. Inter-variant agreement is reported as pairwise Cohen’s κ .

4.2 Results

4.2.1 Per-variant quality against the reference

Variant	Precision	Recall	F1	ρ_0 (false-alarm)	ρ_1 (miss)	MCC	κ	TP	FP	FN	TN
B-SZZ	0.186	0.641	0.288	0.263	0.359	0.232	0.179	1,495	6,565	837	18,422
AG-SZZ	0.186	0.467	0.266	0.192	0.533	0.188	0.163	1,090	4,786	1,242	20,201
MA-SZZ	0.183	0.482	0.265	0.201	0.518	0.188	0.161	1,124	5,030	1,208	19,957
L-SZZ	0.272	0.267	0.269	0.067	0.733	0.202	0.202	622	1,665	1,710	23,322
R-SZZ	0.232	0.300	0.262	0.093	0.700	0.185	0.183	699	2,316	1,633	22,671
RA-SZZ	0.184	0.438	0.259	0.181	0.562	0.178	0.158	1,022	4,531	1,310	20,456

Two observations dominate.



Phase 1 noise profile — precision and recall per variant, and the false-alarm against miss trade-off

Figure 4.1 — Reading the figure. *Left:* each variant’s precision (how much of what it flags is right) beside its recall (how much of what exists it catches). The dashed line marks the 27.2% precision ceiling no variant exceeds. *Right:* each variant plotted by false-alarm rate ρ_0 against miss rate ρ_1 . **The points do not cluster — they spread along a diagonal**, which is the visual signature of a trade-off rather than of random error.

A precision ceiling at 27.2%. No variant exceeds it. Between 72.8% and 81.5% of every commit any variant flags as defect-introducing is not one, judged against the reference. The two decades of refinement separating B-SZZ from RA-SZZ have not produced a variant whose positive predictions are more often right than wrong.

Noise is asymmetric, and the asymmetry differs by variant. B-SZZ is false-positive-heavy: it flags 8,060 commits to capture 1,495 real ones, achieving the highest recall (0.641) at a false-alarm rate of 26.3%. L-SZZ and R-SZZ are the mirror image: false-alarm rates of 6.7% and 9.3%, but they miss 73.3% and 70.0% of real defect-introducing commits respectively. The annotation-graph family sits between.

This asymmetry is the single most consequential fact in this report. **Modelling SZZ noise as symmetric random label flips — the standard assumption in the learning-with-noisy-labels literature — misrepresents it entirely.** A variant that misses seven defects in ten and one that raises four false alarms for every true one are different problems, and whether they have different downstream consequences is precisely what the later phases are designed to establish.

No variant dominates. Ranking by F1 compresses the differences into a band of 0.259–0.288 and hides the trade-off completely. Ranking by MCC puts B-SZZ first (0.232) and RA-SZZ — the most refined variant — last (0.178). Ranking by κ reverses the top: L-SZZ first (0.202), B-SZZ fourth. **The choice of summary metric determines which variant appears best**, which is itself a finding about how SZZ variants are compared in the literature.

4.2.2 Inter-variant agreement

Pairwise Cohen’s κ between the six variants:

	B-SZZ	AG-SZZ	MA-SZZ	L-SZZ	R-SZZ	RA-SZZ
B-SZZ	—	0.690	0.695	0.330	0.427	0.630
AG-SZZ	0.690	—	0.933	0.467	0.587	0.859
MA-SZZ	0.695	0.933	—	0.477	0.596	0.883
L-SZZ	0.330	0.467	0.477	—	0.636	0.477
R-SZZ	0.427	0.587	0.596	0.636	—	0.566
RA-SZZ	0.630	0.859	0.883	0.477	0.566	—

Set against the agreement of each variant with the reference — **κ between 0.158 and 0.202** — the structure is stark. The annotation-graph family agrees with itself at κ 0.859–0.933, which conventional interpretation calls *almost perfect*. The same variants agree with the reference at $\kappa \approx 0.16$, which the same convention calls *slight*.

Variants agree with each other four to six times more strongly than any of them agrees with the reference.

They are not independent estimators converging on a common truth. They share a blame step, a set of assumptions, and therefore a set of failure modes.

The methodological implication is direct and should be stated in any paper that uses SZZ. **High inter-tool agreement has been treated in the literature as evidence of correctness. On this corpus it is evidence of shared error.** Two procedures that make the same mistakes will agree precisely where both are wrong.

4.2.3 Per-project variance

Aggregate figures conceal substantial per-project spread:

Variant	Precision (min / median / max)	Recall (min / median / max)
B-SZZ	0.056 / 0.199 / 0.386	0.220 / 0.700 / 0.919
AG-SZZ	0.053 / 0.222 / 0.325	0.074 / 0.389 / 0.819
MA-SZZ	0.053 / 0.188 / 0.325	0.098 / 0.417 / 0.819
L-SZZ	0.072 / 0.271 / 0.491	0.080 / 0.245 / 0.432
R-SZZ	0.058 / 0.213 / 0.450	0.061 / 0.250 / 0.552
RA-SZZ	0.042 / 0.213 / 0.328	0.110 / 0.361 / 0.789

B-SZZ’s recall ranges from 0.220 to 0.919 across projects — a span of seven-tenths. AG-SZZ’s ranges from 0.074 to 0.819. **A single corpus-level noise rate is a poor description of any individual project**, which is why every statistical test in this report is paired at the project level rather than pooled over commits, and why any later noise injection must be calibrated per project rather than globally.

4.3 The bias model

The later phases require a parametric description of each variant’s noise, and estimating it is the second deliverable of Phase 1. We adopt the standard class-conditional formulation, in which a variant is treated as a noisy channel applied to the reference labels:

- ρ_0 = P(variant labels a commit defective | the reference labels it clean) — the **false-alarm rate**
- ρ_1 = P(variant labels a commit clean | the reference labels it defective) — the **miss rate**

Variant	ρ_0	ρ_1	Character
B-SZZ	0.263	0.359	False-positive-heavy
MA-SZZ	0.201	0.518	Intermediate
AG-SZZ	0.192	0.533	Intermediate
RA-SZZ	0.181	0.562	Intermediate
R-SZZ	0.093	0.700	False-negative-heavy
L-SZZ	0.067	0.733	False-negative-heavy

These six pairs are serialised to `phase1_bias.json`. They are the calibration points for the noise-injection profiles Phase 3 will use: B-SZZ supplies an FP-heavy profile, RA-SZZ an intermediate one, and L-SZZ an FN-heavy one.

Two properties of this channel are worth stating explicitly.

First, $\rho_0 + \rho_1$ approaches 0.8 for the conservative variants. Loss-correction methods that invert the noise channel become numerically unstable as $\rho_0 + \rho_1 \rightarrow 1$, which is a constraint any later mitigation built on these rates will have to respect.

Second, because the positive class is only 8.54% of the corpus, **equal flip rates do not imply equal numbers of flipped labels**. B-SZZ’s ρ_0 of 0.263 applied to 24,987 clean commits produces 6,565 false positives; L-SZZ’s ρ_1 of 0.733 applied to 2,332 defective commits produces 1,710 false negatives. This asymmetry between *rate* and *volume* is not a technicality. It means that a variant with the higher flip *rate* may none the less contribute fewer wrong labels, and any account of which SZZ error actually costs a learner will have to separate the two.

4.3.1 Where the false negatives come from

The conservative variants miss 70–73% of reference-labelled defects. A natural explanation is structural: that line-tracking heuristics cannot blame code which was never written, so defects introduced by omission are invisible.

That explanation is unavailable here. Because the reference labels are themselves blame output (§4.1.1), every reference positive is blame-reachable from some fix line *by construction*. No false negative measured against this reference can be caused by blame being structurally unable to trace a defect. The limitation is a property of the corpus and must be declared, not assumed away.

Decomposing each variant’s false negatives against B-SZZ, the most permissive variant, gives the measured alternative:

Variant	Total FN	Also missed by B-SZZ	Found by B-SZZ, then filtered away
AG-SZZ	1,242	765 (62%)	477 (38%)
MA-SZZ	1,208	763 (63%)	445 (37%)
RA-SZZ	1,310	765 (58%)	545 (42%)
R-SZZ	1,633	808 (49%)	825 (51%)
L-SZZ	1,710	807 (47%)	903 (53%)

Two mechanisms of comparable weight:

1. **Seed-line divergence (47–63%).** Even B-SZZ, which blames every modified line, misses 837 reference positives. Both procedures run `git blame`; they disagree because they blame *different lines*. B-SZZ traces from all modified lines, the reference from the subset three annotators verified. The two land on different commits.
2. **Self-inflicted filtering (37–53%).** Commits B-SZZ identified correctly, which the variant’s own refinement then discarded. For the two most conservative variants this is the *larger* component: R-SZZ discards 55.2% and L-SZZ 60.4% of the reference defects B-SZZ had already found.

The refinements are the problem, not the blame step. Each variant was designed to remove B-SZZ’s false positives, and each succeeds — ρ_0 falls from 0.263 to 0.067. But more than half of what the most aggressive filters remove is correct. This is a precision/recall trade made badly, and it is measurable precisely *because* both sides share a blame step.

The variants are not strict subsets of B-SZZ: between 4.7% (R-SZZ) and 11.9% (RA-SZZ) of their flags lie outside it, so they differ in candidate-selection strategy as well as in filtering.

4.4 Discussion — answering RQ1

How much do the variants disagree with each other? Substantially, and structurally. The annotation-graph family (AG/MA/RA) forms a tight cluster at κ 0.859–0.933. L-SZZ and R-SZZ form a second, looser cluster (κ 0.636). Agreement *between* clusters falls to κ 0.330–0.596. The variants partition into two design philosophies — filter the candidate set, or select a single candidate from it — and the partition is visible in the agreement matrix.

How much do they disagree with the reference? Far more than with each other. κ 0.158–0.202 against the reference versus up to 0.933 among themselves. This is the chapter’s central negative result and it undermines a common methodological practice: cross-variant agreement cannot be used to argue that a labelling is correct.

What is the direction of each variant’s bias? Bifurcated, and predictable from the variant’s design. B-SZZ over-flags because it blames indiscriminately. L-SZZ and R-SZZ under-flag because each collapses a candidate set to a single commit, discarding the rest. The annotation-graph family occupies the middle because it filters without collapsing. **Refinement has not reduced error; it has relocated error from the false-positive column to the false-negative column.**

That relocation is defensible only if false positives are more costly than false negatives to whatever consumes the labels. **That premise is untested in the literature and is not tested here;** Phase 3 is designed to test it directly, and §6.2 states why the question is not answerable from Phase 2’s evidence alone.

4.4.1 What this chapter licenses, and what it does not

Licensed. That SZZ labels on this corpus are noisy at rates far exceeding those usually assumed; that the noise is class-conditional rather than symmetric; that variants share failure modes; that measured noise is a lower bound on true noise.

Not licensed. Any statement that a particular variant is “wrong” in an absolute sense — the reference is a blame-derived construct, not truth. Any generalisation beyond Apache Java projects of this dataset lineage. Any claim about defects introduced purely by deletion, which the corpus excludes by construction.

The next chapter asks the question Phase 1 cannot: whether noise of this magnitude and shape actually changes what a defect prediction model appears to achieve.

Chapter 5 — Phase 2: Downstream Impact Under Honest Evaluation

Chapter 4 established that SZZ labels are noisy, that the noise is class-conditional, and that variants share failure modes. None of that tells us whether the noise *matters*. A model might be robust to it; a metric might be insensitive to it; the effect might be smaller than the variance between projects.

This chapter answers **RQ2**: how does the choice of SZZ label source shift measured JIT-SDP performance across evaluation regimes of increasing realism?

Its central finding is not the one the research question anticipates. The label source matters, but **the evaluation protocol matters more**, and the largest single effect in the entire study is an artefact of how the field scores its models rather than of which labels it trains them on.

5.1 Method

5.1.1 The design

Phase 2 is a four-way factorial grid.

Dimension	Levels
Model	LApredict, JITLine, ORB
Training label source	Reference labels + six SZZ variants (7)
Evaluation regime	Naive k -fold, chronological, chronological-online, prequential-with-latency (4)
Scoring convention	Oracle-scored, self-scored (2)

Twenty-one projects, ten random seeds, giving **16,380 evaluation records**.

Models. LApredict is logistic regression on a single feature — lines added — included because prior work has found such baselines competitive with deep models, making it a test of whether measured effects are properties of the task or of model capacity. JITLine is a 100-tree random forest with SMOTE oversampling and threshold moving, representing the high-capacity batch state of the practice. ORB is a 20-member online logistic ensemble with Poisson oversampling whose rate responds to the model’s recent prediction bias; it is the only model of the three that can operate on a stream.

Regimes, in increasing order of realism:

1. **Naive k -fold** — random 10-fold cross-validation, ignoring commit order. Included precisely because it is indefensible: a model may train on 2020 commits and be tested on 2015 ones. It is the regime most widely used in the JIT-SDP literature, which is why it is the baseline rather than the strawman.
2. **Chronological** — a single 50/50 split in commit-timestamp order. Removes temporal leakage; retains the assumption that all training labels are available at training time.
3. **Chronological-online** — ORB trained sequentially on the first half, then frozen and evaluated on the second. Isolates the learner change from the regime change.
4. **Prequential with verification latency** — test-then-train streaming. Each commit is predicted when it arrives, and its label is delivered only when it would actually have become known.

5.1.2 The scoring convention, and why it is the experiment

The distinction between *oracle-scored* and *self-scored* is the conceptual core of this chapter.

- **Self-scored** — the model is evaluated against the same noisy SZZ labels it was trained on. **This is what the JIT-SDP literature does.**
- **Oracle-scored** — the model is trained on SZZ labels but evaluated against the independently constructed reference labels, held out and never used in training.

Self-scoring measures how faithfully a model reproduces a heuristic. Oracle-scoring measures how well it identifies what the reference calls defect-introducing. These are different quantities, and the field reports the first while interpreting it as the second. Holding out an independently sourced scoring key makes the gap between them measurable, and measuring that gap is the purpose of Phase 2.

5.1.3 Verification latency

Under the prequential regime, a defect label is not available when the commit arrives; it becomes available when the defect is found and fixed. The timeline implemented is:

- At commit time t : the model predicts; the prediction is scored immediately against the reference label.
- If the training label is clean: delivered as a training example at $t + W$, with $W = 90$ days.
- If defective and the fix arrives within the window: delivered as a positive at the fix time.

- If defective but the fix arrives later: delivered **first as a negative at $t + W$, then corrected to positive at the fix time.**
- If defective with no linked fix: delivered as a negative at $t + W$ and never corrected.

The third case is what makes latency more than a delay. A model operating under a 90-day window does not merely wait for labels — **it is actively trained on labels it will later learn were wrong.** On this corpus, 53.0% of defect labels arrive after the window, with a median latency of 113 days.

5.1.4 Statistical protocol

Every test is paired at the **project** level, $n = 21$, with seeds averaged before pairing. Pooling over the 16,380 records would be pseudo-replication: the seeds are not independent observations of anything.

Each comparison reports a Wilcoxon signed-rank p -value, a **matched-pairs rank-biserial correlation** (the proportion of projects moving in the effect’s direction, derived from the same signed ranks as the p -value), a **Hodges–Lehmann** location estimate, and a percentile bootstrap confidence interval computed for the Hodges–Lehmann statistic itself.

Seventy-four tests were run. Multiplicity is corrected twice: **within test family**, and **globally across all 74**. The global column exists because the families were defined during analysis rather than declared in advance, so a sceptic could argue the grouping was chosen to favour the results. A claim surviving global Holm needs no defence of how families were drawn. Thirty-two tests survive within-family correction; **twenty-one survive global correction**, and it is the global column that the headline claims rest on.

Both phases reported here are exploratory and are labelled as such throughout. No hypothesis in this report was pre-registered, which is why the global multiplicity correction is reported alongside the within-family one.

5.2 Results

5.2.1 The full matrix

Mean MCC over 21 projects $\times$ 10 seeds.

Batch models

Model	Labels	<i>k</i> -fold oracle-scored	<i>k</i> -fold self-scored	Chrono oracle-scored	Chrono self-scored
JITLine	reference	0.2430	—	0.1016	—
JITLine	B-SZZ	0.1751	0.4129	0.1324	0.2232
JITLine	AG-SZZ	0.1164	0.3037	0.0904	0.1351
JITLine	MA-SZZ	0.1272	0.3295	0.0786	0.1395
JITLine	L-SZZ	0.1391	0.2434	0.0804	0.1482
JITLine	R-SZZ	0.1131	0.1801	0.0626	0.0920
JITLine	RA-SZZ	0.1164	0.3070	0.0857	0.1402
LApredict	reference	0.2058	—	0.1734	—
LApredict	B-SZZ	0.1941	0.3510	0.1634	0.2918
LApredict	AG-SZZ	0.2003	0.2472	0.1685	0.1975
LApredict	MA-SZZ	0.1995	0.2626	0.1664	0.2003
LApredict	L-SZZ	0.2078	0.2679	0.1706	0.2174
LApredict	R-SZZ	0.2016	0.1779	0.1731	0.1666
LApredict	RA-SZZ	0.2000	0.2603	0.1679	0.2143

Reference-trained cells have no separate self-scored value: the two conventions coincide by construction.

Online model (ORB)

Labels	Chrono-online oracle	Chrono-online self	Prequential (time-avg)	Prequential (terminal)
reference	0.0777	—	0.0970	0.0685
B-SZZ	0.0767	0.1613	0.0578	0.0566
L-SZZ	0.0082	0.0695	0.0582	0.0209
RA-SZZ	0.0193	0.0673	0.0395	0.0184
AG-SZZ	0.0177	0.0684	0.0365	0.0125
MA-SZZ	0.0216	0.0717	0.0353	−0.0030
R-SZZ	0.0114	0.0349	0.0315	0.0104

Two features of this table are worth marking before the analysis.

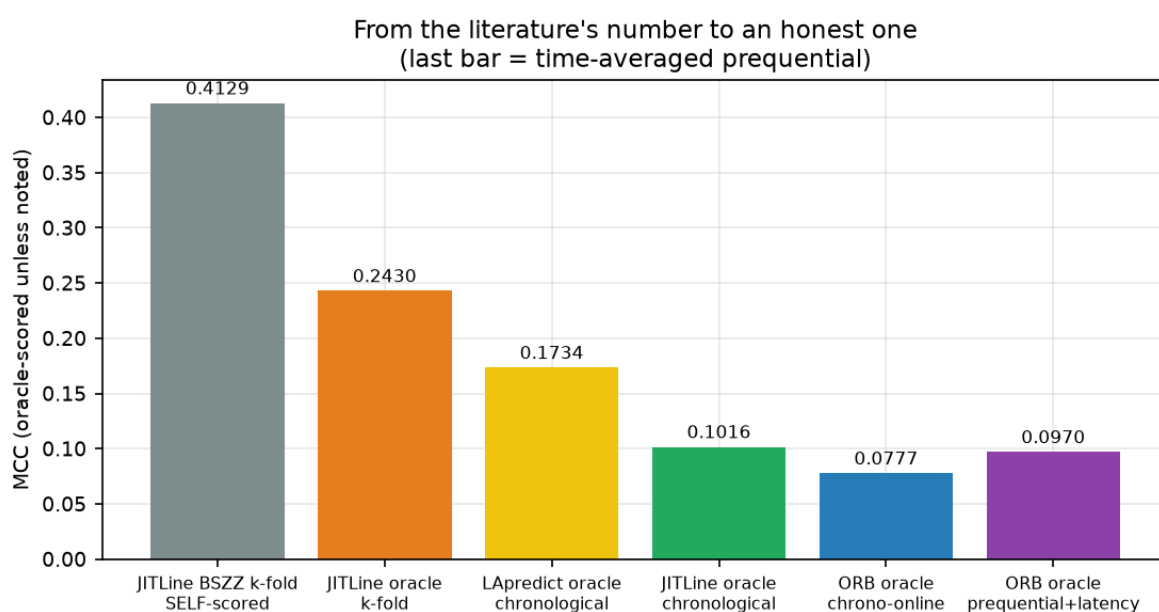
First, **LApredict** — **logistic regression on one feature** — **is the best model in the corpus under a chronological split** (0.1734 against JITLine’s 0.1016 on reference labels). The random forest’s advantage exists only under *k*-fold.

Second, MA-SZZ’s apparently harmful negative MCC (−0.0030) exists only under the terminal estimator; under the time-averaged estimator it is +0.0353. **No MCC is quoted anywhere in this report without naming the estimator**, and §5.2.5 explains why.

5.2.2 The inflation ladder

Step	Configuration	MCC	What was removed
0	JITLine, B-SZZ labels, k -fold, self-scored	0.4129	— (the literature's configuration)
1	JITLine, B-SZZ labels, k -fold, oracle-scored	0.1751	Circular self-scoring (-0.238)
2	JITLine, reference labels, chronological	0.1016	Temporal leakage
3	ORB, reference labels, prequential + latency	0.0970	Batch-to-stream

A number that reads as **0.41** under the field's standard protocol is **0.10** under a defensible one.



The inflation ladder — from the literature's configuration to a defensible one

Figure 5.1 — Reading the figure. Each bar is the same prediction task measured under progressively more honest conditions, left to right. The grey bar is what the field's protocol reports; the rightmost bar is what survives realistic evaluation. **The height difference between the first and last bar is this report in one picture.**

The ladder must be read as **descriptive, not causal**. Step 3 changes the learner, the regime and the evaluation window simultaneously. §5.3 explains why an earlier attempt to decompose it was withdrawn.

5.2.3 Result 1 — Temporal leakage inflates, in proportion to model capacity

Random k -fold against a chronological split, paired by project. Fourteen comparisons: seven label sources $\times$ two batch models.

Model	Labels	k-fold	Chrono	HL	95% CI	Rank-biserial	Holm (family)	Holm (global)
JITLine	reference	0.2430	0.1016	+0.1373	[+0.1046, +0.1721]	+1.000	1.3e-05	7.1e-05
JITLine	L-SZZ	0.1391	0.0804	+0.0667	[+0.0216, +0.0900]	+0.792	0.0094	0.0389
JITLine	R-SZZ	0.1131	0.0626	+0.0515	[+0.0258, +0.0764]	+0.706	0.0393	0.1607
LApredict	reference	0.2058	0.1734	+0.0339	[+0.0006, +0.0647]	+0.489	0.4015	1.000

(Four of fourteen shown; the remainder follow the same pattern.)

The headline is JITLine on reference labels: **+0.1373 MCC, with a rank-biserial of exactly +1.000 — all 21 projects, without exception.** The exact test saturates.

The effect scales with model capacity. JITLine loses 0.137; LApredict loses 0.034 and does not survive correction. A single-feature logistic regression has little capacity to exploit temporal leakage; a 100-tree forest has a great deal.

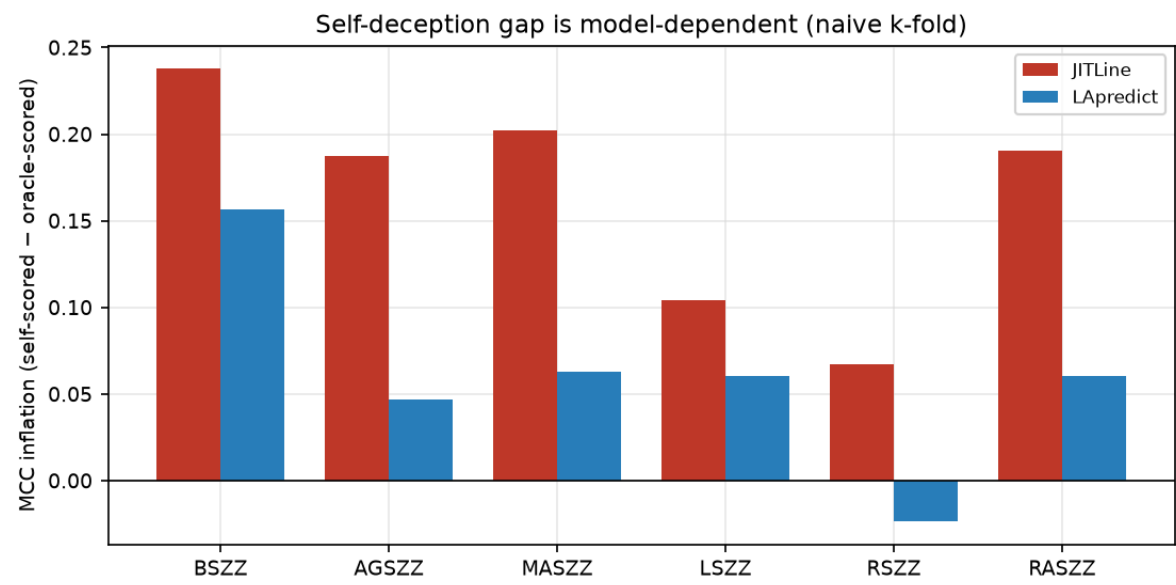
This is the most damaging form the finding could take. The inflation is largest exactly where the field reports its strongest results, so **the apparent superiority of high-capacity models over simple baselines is partly an artefact of the evaluation protocol that measures it.**

5.2.4 Result 2 — Self-scoring inflates further, and it is model-dependent

Thirty-six comparisons: for each (model, label source, regime), the same predictions scored against the training labels versus against the reference.

Model	Labels	Regime	Self	Oracle	HL	95% CI	Rank-biserial	Holm (global)
JITLine	B-SZZ	k-fold	0.4129	0.1751	+0.2303	[+0.1886, +0.2784]	+0.991	1.0e-04
JITLine	MA-SZZ	k-fold	0.3295	0.1272	+0.1985	[+0.1393, +0.2648]	+1.000	1.0e-04
JITLine	RA-SZZ	k-fold	0.3070	0.1164	+0.1861	[+0.1221, +0.2523]	+1.000	1.0e-04
JITLine	AG-SZZ	k-fold	0.3037	0.1164	+0.1797	[+0.1201, +0.2456]	+1.000	1.0e-04
LApredict	B-SZZ	k-fold	0.3510	0.1941	+0.1549	[+0.1213, +0.2029]	+0.957	6.0e-04
LApredict	AG-SZZ	k-fold	0.2472	0.2003	+0.0510	[−0.0123, +0.1100]	+0.351	1.000
LApredict	R-SZZ	k-fold	0.1779	0.2016	−0.0238	[−0.0662, +0.0265]	−0.221	1.000

+0.2303 MCC is the largest effect in this report. It is also the effect with the clearest methodological reading: a random forest trained on B-SZZ labels appears roughly two and a half times better at finding defects than it is, because it is being graded by the heuristic that taught it.



Self-scoring gap by variant and model

Figure 5.2 — Reading the figure. Each bar is how far a model’s apparent score *drops* when it stops being graded against the heuristic that trained it and is graded against the reference instead. Red is JITLine (high capacity), blue LAPredict (low capacity). **The red bars are consistently much taller** — the gap is a capacity phenomenon.

The gap is model-dependent, and the pattern is diagnostic. Under JITLine it is large and significant for every one of the six variants. Under LAPredict it is significant only for B-SZZ, and for R-SZZ it is *negative*. A single-feature logistic regression cannot memorise a heuristic’s idiosyncrasies; a high-capacity forest can. **The self-scoring gap is a capacity phenomenon, which is exactly what one would predict if the mechanism is a model fitting the error structure of its labels.**

The mechanism, tested directly. That reading was, until recently, an interpretation the gap was *consistent with* rather than one it established. We therefore tested it: restricting to the 8,060 B-SZZ-flagged commits and asking whether a model can separate B-SZZ’s false positives from its true positives using the same 14 features.

	Mean AUC
Random forest, chronological split per project	0.599
Same pipeline, training labels permuted (the null)	0.514

Paired across projects: **HL +0.088, CI [+0.041, +0.131], 17/21 projects, *p* = 0.0016.**

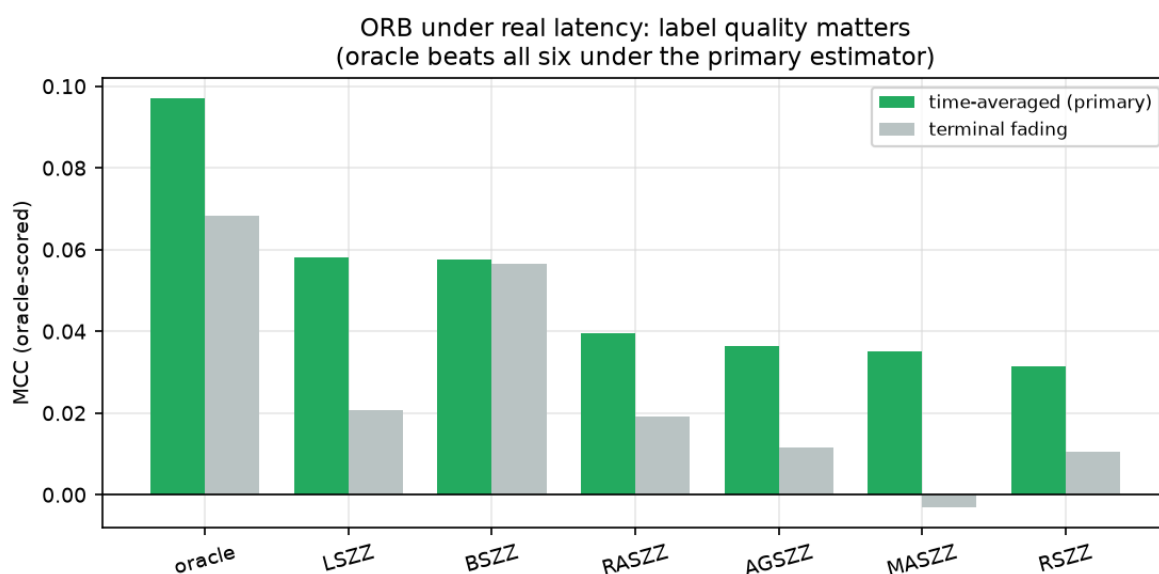
SZZ’s errors carry structure the features can detect, so a model *can* fit them. The mechanism is real. **It is also weak — an AUC of 0.60 is modest separability — and the +0.230 gap is almost certainly not explained by this structure alone.** The claim this report makes is “the error structure is learnable”, not “SZZ’s mistakes are easy to predict”.

5.2.5 Result 3 — Under realistic streaming, the label source matters

Under the prequential regime with real verification latency, ORB trained on reference labels against ORB trained on each variant, all scored against the reference.

Reference vs	Mean gap	HL	95% CI	Rank-biserial	Projects	Holm (family)	Holm (global)
R-SZZ	+0.0655	+0.0603	[+0.0418, +0.0816]	+0.948	19/21	0.0001	0.0009
MA-SZZ	+0.0617	+0.0638	[+0.0390, +0.0845]	+0.896	18/21	0.0003	0.0042
AG-SZZ	+0.0605	+0.0600	[+0.0290, +0.0851]	+0.818	17/21	0.0009	0.0243
RA-SZZ	+0.0575	+0.0577	[+0.0357, +0.0808]	+0.879	18/21	0.0004	0.0064
B-SZZ	+0.0392	+0.0412	[+0.0219, +0.0521]	+0.879	18/21	0.0004	0.0064
L-SZZ	+0.0388	+0.0341	[+0.0140, +0.0552]	+0.723	16/21	0.0025	0.1241

All six intervals exclude zero; **five of six survive global Holm correction.**



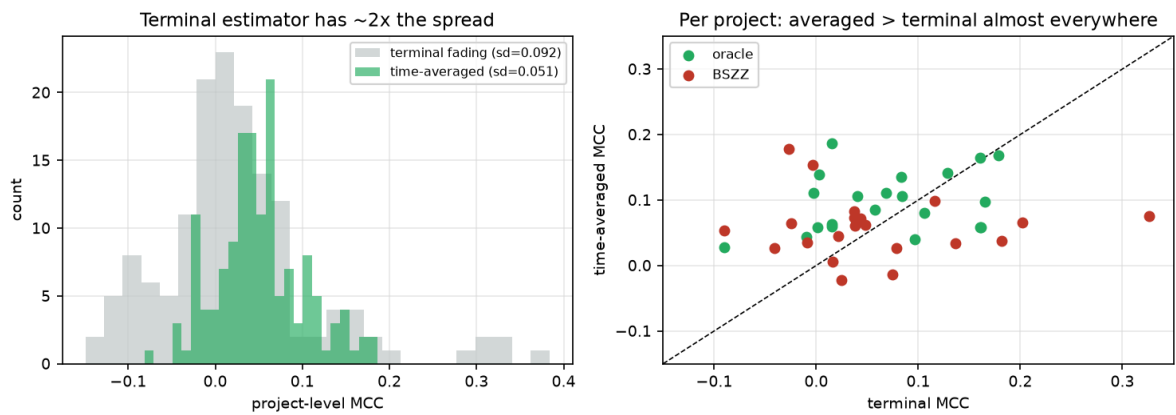
Label source comparison under both prequential estimators

Figure 5.3 — Reading the figure. Green bars are the primary time-averaged estimator; grey the secondary terminal estimator. The reference labels are leftmost and tallest under both. Note that MA-SZZ dips below zero under the grey bars only — an artefact of the noisier statistic, not a model that actively harms.

The B-SZZ comparison is the most informative row. Because the reference and B-SZZ share the same blame step and differ only in which lines seed it (§4.1.1), this contrast is a near-controlled ablation of tangled commits. Its value is **+0.041 MCC, CI [+0.022, +0.052]** — a defensible estimate of what knowing which lines in a fix actually fix the bug is worth to a deployed online learner.

The gap does not track recall. The correlation between a variant’s oracle-versus-variant gap and its recall is -0.18 . L-SZZ (recall 0.267) and B-SZZ (recall 0.641) have almost identical gaps. Whatever drives the penalty, it is not simply how many defects the labeller finds. This report cannot say what does drive it: §6.2 names the experiment that would.

Estimator dependence, reported rather than buried. Two summaries of the prequential MCC trajectory are available: a *terminal* value computed from a fading confusion matrix (effective window ≈ 100 commits), and the *time-averaged* mean of the trajectory. They agree on every comparison in this report except one: under the terminal estimator the B-SZZ row alone loses significance. The time-averaged estimator is treated as primary on a measured basis — roughly half the project-level variance (0.051 against 0.092) — and **not** by appeal to a standard. MCC is not a decomposable loss, so the prequential-with-fading construction does not extend to it directly; both summaries are ad-hoc, and this report says so rather than claiming canonical status for either.



Why the two estimators differ — variance and per-project comparison

Figure 5.4 — Reading the figure. *Left:* the spread of project-level scores under each estimator — the terminal distribution is visibly wider, which is the measured basis for preferring the time-averaged value. *Right:* each project plotted under both; points above the diagonal are projects where the time-averaged value is higher.

5.3 An honest negative—

5.3 An honest negative: the batch-to-stream contrast could not be isolated

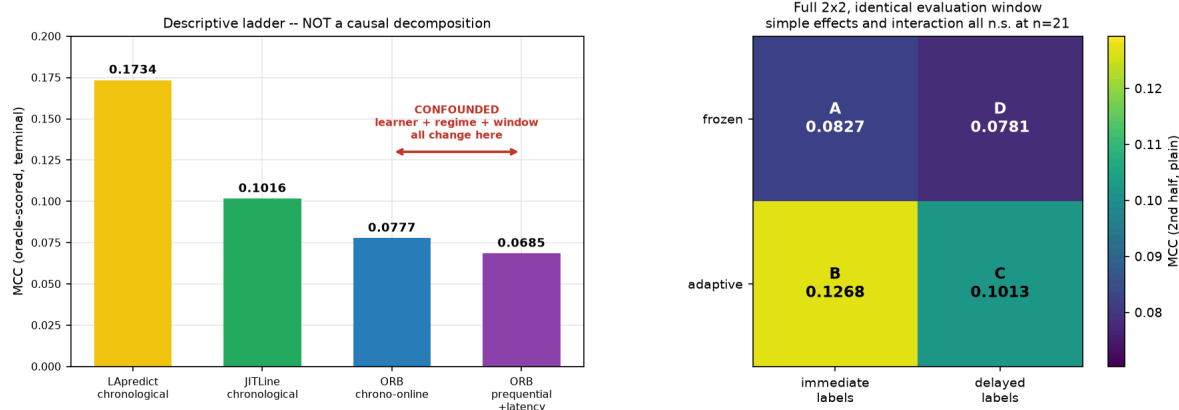
An earlier version of this work reported that verification latency accounts for roughly 9% of the drop from batch to streaming performance. **That claim has been withdrawn, and the withdrawal is reported here rather than removed.**

The comparison was not identified. It changed three things at once: the learner (forest → online ensemble), the label timing (immediate → delayed), and the evaluation window (a held-out second half → the whole stream). Any of the three could produce the observed difference.

A full 2×2 factorial — learner (frozen / adaptive) crossed with labels (immediate / delayed) — was built to isolate the effects:

Contrast	Estimate	Interval
Latency effect	$\approx +0.023$	contains zero
Adaptivity effect	≈ -0.046	contains zero
Interaction	—	contains zero

Every contrast, including the interaction, has a confidence interval spanning zero. With 21 paired projects, effects below roughly 0.04 MCC are not resolvable, and these fall in that range.



The confounded ladder step, and the 2x2 built to separate it

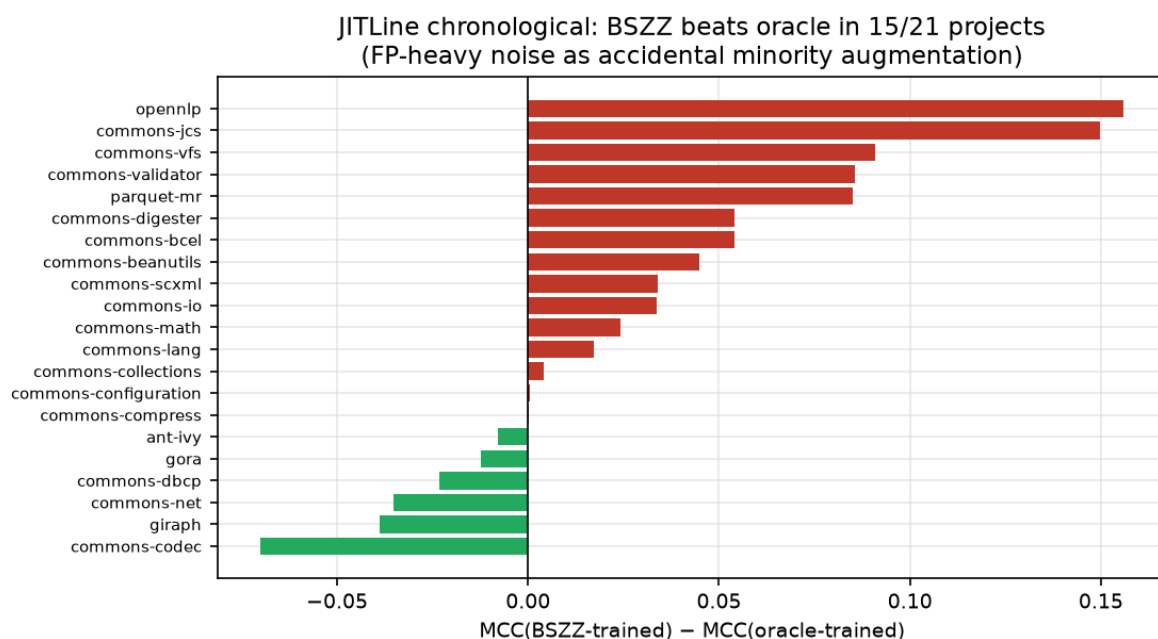
Figure 5.5 — Reading the figure. *Left:* the descriptive ladder with the problematic step marked — learner, regime and evaluation window all change there at once. *Right:* the 2x2 that separates them, brighter meaning higher MCC. The cells are close enough that no contrast resolves.

The honest conclusion is that **21 projects cannot resolve the decomposition**, and it is reported as unresolved. A design that holds the learner fixed by construction — injecting controlled noise rather than switching label sources — would make the question answerable, and that is how Phase 3 approaches it.

5.3b An exploratory finding: the JITLine anomaly

One result in the Phase 2 matrix runs against the chapter’s argument and is reported because it does.

Under chronological evaluation, **B-SZZ-trained JITLine (0.1324) beats reference-trained JITLine (0.1016)**, winning in **15 of 21 projects**. Training on demonstrably noisier labels produces a better model.

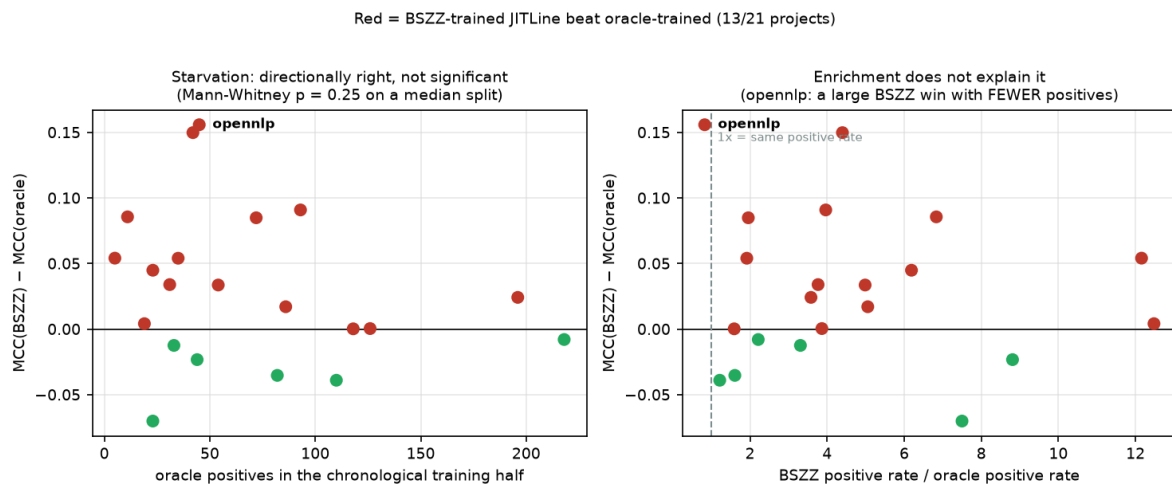


Per-project B-SZZ-minus-reference gap for JITLine

Figure 5.6 — Reading the figure. One bar per project. Bars to the right are projects where training on *noisy* B-SZZ labels beat training on the reference labels; bars to the left are the reverse.

The phenomenon is real, not seed noise. 97.7% of the between-project variance is genuine rather than seed variation; 16 of 21 projects show a gap more than two standard errors from zero; and it survives all four threshold-selection protocols tested.

But no project characteristic predicts where it happens.



What predicts the anomaly — nothing does

Figure 5.7 — Reading the figure. *Left:* the gap against how many defect examples a project supplies for training — if minority starvation were the mechanism, the points would slope clearly downward. *Right:* the gap against how many extra positive labels B-SZZ supplies. `opennlp` is annotated on both panels because it sits on the wrong side of the 1× line.

Predictor	Spearman ρ with gap	p
Training-half defect examples	−0.336	0.14
Total defect examples	−0.216	0.35
Label-arrival coverage	−0.175	0.45
Number of commits	−0.149	0.52
B-SZZ precision	−0.135	0.56
B-SZZ recall	−0.108	0.64
Enrichment (B-SZZ rate ÷ reference rate)	+0.066	0.78
Reference defect rate	−0.056	0.81
B-SZZ flag rate	−0.029	0.90

Not one reaches even uncorrected significance, across 18 tests where roughly one would be expected to by chance.

Two mechanisms point the right way without reaching significance. *Minority enrichment:* projects with few training defects gain +0.0479 against +0.0015 for positive-rich projects (Mann–Whitney $p = 0.245$). *Headroom:* B-SZZ helps most where the reference-trained model is already weak (reference MCC 0.093 in B-SZZ-win projects against 0.119

elsewhere, $p = 0.137$). These are not independent — fewer defects *causes* a weaker model — so they are one hypothesis rather than two.

A counter-example the enrichment story cannot absorb. `opennlp` is the second-largest B-SZZ win ($+0.162 \pm 0.010$), yet B-SZZ flags a *lower* rate there than the reference — 6.91% against 8.38%, an enrichment of 0.82×. There is no extra minority mass to explain the gain.

It is a batch phenomenon only. Under streaming, the reference beats B-SZZ in 18 of 21 projects; the anomaly does not carry over, which suggests batch and online learners are sensitive to different properties of the labels — itself a question for the later phases.

How this is stated in this report: the phenomenon is robust and worth reporting; the *mechanism* is not established, and no mechanism is claimed.

5.4 Robustness

Estimator and fading factor. The label-source conclusion was re-examined across fading factors 0.90–0.999 (effective windows 10–1000 commits) crossed with warm-up skips of 0/5/10/25% — 20 settings $\times$ 6 variants = 120 comparisons. In **120 of 120**, the Hodges–Lehmann estimate favours the reference labels, the bootstrap interval excludes zero, and the comparison survives *grid-wide* Holm correction.

This is one result re-examined under different analytical choices, not 120 findings, and grid-wide correction is applied so the framing cannot be mistaken.

Project weighting. Every figure is an unweighted mean over projects carrying between 19 and 335 reference defects; eight of 21 carry fewer than 25 in the chronological test half. Re-running the headline contrasts with the smallest projects excluded:

Claim	All 21	≥ 25 defects ($n = 13$)
k -fold inflation	+0.137	+0.136
Self-scoring gap (k -fold)	+0.230	+0.232
Label source (reference vs B-SZZ)	+0.041	+0.043

No estimate moves by more than 0.002 MCC and all three still survive correction. **The conclusions are not artefacts of the small projects.** One claim does weaken: the self-scoring gap under the *chronological* regime falls from +0.098 to +0.075 and loses significance — not the headline, but the deployment-relevant regime, and reported here rather than omitted.

5.5 Discussion — answering RQ2

The label source shifts measured performance, and the shift is significant under every regime that permits a fair test. Under realistic streaming the penalty for training on SZZ rather than the reference is 0.034–0.064 MCC, against a deployable baseline of roughly 0.097 — between a third and two-thirds of the available signal.

But the regime shifts it more. Moving from k -fold to chronological costs JITLine 0.137. Removing circular scoring costs a further 0.230. **The two protocol effects together are roughly nine times the label-source effect.**

This inverts the question the chapter set out to ask. Phase 2 was designed to measure what SZZ noise costs. What it found is that **the cost of SZZ noise is small relative to the cost of the field's evaluation conventions** — and that the two interact, because self-scoring is only possible *because* the labels are heuristic. If the field had ground truth, it would score against it, and the +0.230 gap could not exist.

The implication for reading published results. A JIT-SDP result reported under random k -fold with self-scored SZZ labels is not a measurement of defect-prediction capability. On this corpus the difference between that protocol and a defensible one is 0.41 versus 0.10 MCC. Any comparison between published models measured this way is confounded by capacity-dependent inflation, since the artefact is larger for models with more capacity to exploit it.

What Phase 2 could not determine. Which *kind* of SZZ error does the damage. The label-source gap shows that label quality matters; it does not show whether the cost comes from the false positives SZZ invents or the defects it misses. The gap's failure to track recall (−0.18) rules out the simplest explanation and leaves the question open.

That question is the subject of Phase 3, and answering it requires an intervention rather than an observation. Chapter 6 sets out the design.

Chapter 6 — Conclusions and Planned Work

This report covers the first two phases of the project. This chapter states what they establish, what they do not, and how the remaining phases are designed — including two commitments made here, in advance, rather than after the data are seen.

6.1 What Phases 1 and 2 establish

6.1.1 The two research questions answered

RQ1 — How much do SZZ variants disagree, and in which direction? Substantially, and structurally. No variant exceeds 27.2% **precision** against the reference. Variants agree with one another at κ up to **0.933** while agreeing with the reference at κ **0.158–0.202** — four to six times more strongly with each other than with the thing they are estimating. Bias is bifurcated: B-SZZ over-flags ($\rho_0 = 0.263$), while L-SZZ and R-SZZ under-flag ($\rho_1 = 0.733$ and 0.700). **Two decades of refinement have relocated error rather than reduced it**, moving it from the false-positive column to the false-negative column.

RQ2 — How does the label source shift measured performance across regimes? It shifts it significantly — 0.034 to 0.064 MCC under realistic streaming, against a deployable baseline near 0.097. **But the evaluation protocol shifts it far more.** Random k -fold inflates a high-capacity model by **+0.137 MCC**; scoring against the training labels inflates it by a further **+0.230**. The two protocol effects together are roughly nine times the label-source effect, and both scale with model capacity.

6.1.2 The finding that reframes the project

The project set out to measure what SZZ noise costs. It found that **the cost of SZZ noise is small relative to the cost of the field's evaluation conventions** — and that the two are not independent.

Self-scoring is only possible *because* the labels are heuristic. If the field had ground truth, it would score against it, and the +0.230 gap could not exist. **The labelling failure creates the evaluation failure.** A result reported under random k -fold with self-scored SZZ labels is not a measurement of defect-prediction capability, and on this corpus the difference between that protocol and a defensible one is 0.41 against 0.10 MCC.

6.1.3 Practical significance, stated honestly

The measurement critique is statistically overwhelming — rank-biserial +1.000 on the leakage effect, 21 of 21 projects, surviving global correction. **Practical significance is more modest and should be reported as such.** Even with the reference labels and an honest protocol, deployable MCC on this corpus is approximately **0.097**.

This report does not claim that better labels would make JIT-SDP good. It claims the field’s reported numbers overstate what it currently achieves by roughly a factor of four. The low ceiling is itself a finding: a field reporting 0.41 and delivering 0.10 has a calibration problem independent of whether 0.10 is useful.

6.1.4 The reconciliation a reader will otherwise trip over

Studies that evaluate SZZ against developer-informed oracles report precision in the range 0.39–0.66. This report reports 0.186–0.272. **The two are not in conflict, and the difference is the denominator.**

Those studies score *conditionally* on bug-fixing commits already known to have an inducing commit: given a defect, which earlier commit introduced it? This report scores over an entire project history of 27,319 commits, the overwhelming majority of which are clean: is *this* commit, drawn from all of history, defect-introducing? A variant right two times in three on the first question can be wrong four times in five on the second. **Any comparison between the two literatures has to reconcile the denominator first.**

6.2 What Phases 1 and 2 do not establish

Three questions are raised by this report and are not answerable from its evidence. Each is stated with the design that would answer it.

6.2.1 Which SZZ error actually costs the learner

Phase 2 shows that label quality matters under streaming. It does **not** show whether the cost comes from the false positives SZZ invents or from the defects it misses. The simplest explanation is ruled out: the penalty does not track a variant’s recall ($\rho = -0.18$), and L-SZZ and B-SZZ have almost identical gaps despite recalls of 0.267 and 0.641.

The question cannot be settled by comparing label sources, because whole labelling procedures differ in many ways at once. **It needs an intervention.** Phase 3’s design takes real B-SZZ labels and uses the reference to correct *one error type at a time* — removing the false positives, or restoring the false negatives — holding the stream, the learner and the latency model fixed. The difference between conditions is then attributable to the repair.

Two controls must be built into that design from the start, because this report has already identified the confounds they would otherwise create:

- **Error matching.** B-SZZ carries 6,565 false positives against 837 false negatives, an 8:1 imbalance (§4.2.1). “Repair all of each” therefore conflates *which error is costlier per label* with *which error there is more of*. The design must include a matched-mass condition.
- **Delivery.** A restored false negative has no linked fix and so no natural arrival time. If arrival times are imputed from the empirical latency distribution, a null result cannot distinguish “these labels do not matter” from “these labels cannot arrive in time to matter”. The design must include an immediate-delivery ceiling condition.

6.2.2 Why an online learner should be sensitive at all

ORB handles class imbalance by oversampling at a rate set from the *observed* rate of defect labels — that is, from label **quantity**. It has no access to label **quality**, so a false positive receives the same amplification as a true one.

That makes amplification a plausible route by which SZZ noise damages an online learner, and it makes a measurable prediction: the oversampling rate applied to defect-labelled arrivals should be an inverse function of how many such labels the stream delivers, *regardless of whether they are correct*. **This report does not test that**. Instrumenting the learner's internal rate across a calibrated dose grid would.

6.2.3 Whether the latency effect can be isolated

§5.3 withdraws a decomposition of the batch-to-streaming drop because the contrast was not identified, and reports the replacement 2×2 as unresolved — every contrast, including the interaction, has an interval spanning zero at $n = 21$.

The obstacle is that switching label sources changes the learner's training signal and its evaluation window together. A design that **holds the learner fixed by construction** — injecting controlled noise into a single label source rather than switching between sources — removes that confound. It also requires a true no-latency control: an arm in which *every* label, positive and negative, arrives immediately. A fixed-delay arm is not a no-latency arm, since it delays the 91.5% majority class as heavily as the minority.

6.3 Commitments made in advance

Two are recorded here so that they precede the data rather than follow it.

The mitigation phase will be pre-registered. Hypotheses, primary model, acceptance bar and multiplicity correction will be committed to version control before the first run, and the commit history will timestamp the order. Phases 1 and 2 are exploratory and are labelled as such throughout; the global Holm column is what makes their headline claims defensible without a registration.

The non-degradation gate is the acceptance bar, not the headline test. A mitigation that improves performance on noisy labels while damaging it on clean ones is not adoptable, because a practitioner cannot know in advance which case they are in. That gate will be applied before any effect size is examined.

6.4 Threats to validity

The reference is not ground truth. It is `git blame` seeded with human-verified fix lines (§3.2.2). Comparisons against it isolate the cost of tangled commits, not total labelling error, and **all measured noise is therefore a lower bound**: blame error is present on both sides of every comparison and cancels.

The reference condition's label timing is SZZ-derived. This is the most serious threat in the study. The published labels carry no native fix-to-inducing linkage, so arrival times were reconstructed from the union of SZZ mappings. The reference is blame-derived in construction *and* SZZ-derived in timing. A coverage sensitivity analysis (67.8% → 100%) bounds part of the exposure; the timing itself is not bounded, and every streaming result inherits it.

The corpus is SZZ-shaped. Changes adding no new lines were excluded by the dataset authors, so defects introduced purely by deletion cannot appear. Because the reference is itself blame output, blame-unreachability is not an available explanation for any measured false negative.

Project heterogeneity is measured and does not drive the results. Projects span 544–4,026 commits and 19–335 defects and are weighted equally. Excluding those below a 25-defect floor moves no headline estimate by more than 0.002 MCC, and all headline claims still survive correction (§5.4).

Effects below roughly 0.04 MCC are not resolvable at $n = 21$. This is load-bearing for the latency decomposition, which is reported as unresolved rather than as null.

Nothing here is pre-registered, which is why global multiplicity correction is reported alongside the within-family correction, and why the headline claims are those surviving the global column.

One benchmark family. Twenty-one Apache Java repositories of a single dataset lineage.

6.5 Closing

The report began by asking how much SZZ label noise costs a defect prediction model. The answer — between a third and two-thirds of the available signal under realistic streaming — turned out to be less interesting than a question it exposed along the way: **how much of what the field reports is measurement artefact rather than capability?**

On this corpus, most of it. A number that reads as 0.41 under the field’s standard protocol is 0.10 once the model is scored against something other than the heuristic that trained it and evaluated in an order that time permits. That gap — roughly four-fifths of the reported signal — is quantified here, bounded by confidence intervals, corrected for multiple comparisons, stable across every analytical choice tested, and reproducible from a single commit hash.

What the report cannot yet say is *why*: which of SZZ’s two errors an online learner cannot survive, and whether a learner can be built to withstand it. Those are the next two phases, and §6.2 states both the designs and the controls that this report has already shown they will need.

References

- Benjamini, Yoav, and Yosef Hochberg. 1995. “Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing.” *Journal of the Royal Statistical Society: Series B* 57 (1): 289–300.
- Cabral, George G., and Leandro L. Minku. 2023. “Towards Reliable Online Just-in-Time Software Defect Prediction.” *IEEE Transactions on Software Engineering*.
- Cabral, George G., Leandro L. Minku, Emad Shihab, and Suhaib Mujahid. 2019. “Class Imbalance Evolution and Verification Latency in Just-in-Time Software Defect Prediction.” *Proceedings of the 41st IEEE/ACM International Conference on Software Engineering (ICSE)*.
- Cohen, Jacob. 1960. “A Coefficient of Agreement for Nominal Scales.” *Educational and Psychological Measurement* 20 (1): 37–46.
- Costa, Daniel Alencar da, Shane McIntosh, Weiyi Shang, Uirá Kulesza, Roberta Coelho, and Ahmed E. Hassan. 2017. “A Framework for Evaluating the Results of the SZZ Approach for Identifying Bug-Introducing Changes.” *IEEE Transactions on Software Engineering* 43 (7): 641–57.
- Davies, Steven, Marc Roper, and Murray Wood. 2014. “Comparing Text-Based and Dependence-Based Approaches for Determining the Origins of Bugs.” *Journal of Software: Evolution and Process* 26 (1): 107–39.
- Destefanis, Giuseppe et al. 2026. “An Audit of Machine Learning Experiments on Software Defect Prediction.” *Empirical Software Engineering*, ahead of print. <https://doi.org/10.1007/s10664-025-10797-w>.
- Falessi, Davide et al. 2020. “On the Need of Removing Last Releases of Data When Using or Validating Defect Prediction Models.” *arXiv Preprint arXiv:2003.14376*.

- Gama, João, Raquel Sebastião, and Pedro P. Rodrigues. 2013. "On Evaluating Stream Learning Algorithms." *Machine Learning* 90 (3): 317–46. <https://doi.org/10.1007/s10994-012-5320-9>.
- Han, Bo, Quanming Yao, Xingrui Yu, et al. 2018. "Co-Teaching: Robust Training of Deep Neural Networks with Extremely Noisy Labels." *Advances in Neural Information Processing Systems (NeurIPS)*.
- Herbold, Steffen, Alexander Trautsch, Benjamin Ledel, et al. 2022. "A Fine-Grained Data Set and Analysis of Tangling in Bug Fixing Commits." *Empirical Software Engineering*, ahead of print. <https://doi.org/10.1007/s10664-021-10083-5>.
- Hoang, Thong, Hoa Khanh Dam, Yasutaka Kamei, David Lo, and Naoyasu Ubayashi. 2019. "DeepJIT: An End-to-End Deep Learning Framework for Just-in-Time Defect Prediction." *Proceedings of the 16th International Conference on Mining Software Repositories (MSR)*.
- Hoang, Thong, Hong Jin Kang, David Lo, and Julia Lawall. 2020. "CC2Vec: Distributed Representations of Code Changes." *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*.
- Hodges, J. L., and E. L. Lehmann. 1963. "Estimates of Location Based on Rank Tests." *The Annals of Mathematical Statistics* 34 (2): 598–611.
- Holm, Sture. 1979. "A Simple Sequentially Rejective Multiple Test Procedure." *Scandinavian Journal of Statistics* 6 (2): 65–70.
- Kamei, Yasutaka, Emad Shihab, Bram Adams, et al. 2013. "A Large-Scale Empirical Study of Just-in-Time Quality Assurance." *IEEE Transactions on Software Engineering* 39 (6): 757–73. <https://doi.org/10.1109/TSE.2012.70>.
- Keshavarz, Hossein, and Meiyappan Nagappan. 2022. "ApacheJIT: A Large Dataset for Just-in-Time Defect Prediction." *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*.
- Kim, Sunghun, Thomas Zimmermann, Kai Pan, and Jr. Whitehead E. James. 2006. "Automatic Identification of Bug-Introducing Changes." *Proceedings of the 21st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 81–90. <https://doi.org/10.1109/ASE.2006.23>.
- Mahbub, Parvez, Ohiduzzaman Shuvo, and Mohammad Masudur Rahman. 2023. "Defectors: A Large, Diverse Python Dataset for Defect Prediction." *Proceedings of the 20th International Conference on Mining Software Repositories (MSR)*.
- Matthews, B. W. 1975. "Comparison of the Predicted and Observed Secondary Structure of T4 Phage Lysozyme." *Biochimica Et Biophysica Acta (BBA) — Protein Structure* 405 (2): 442–51. [https://doi.org/10.1016/0005-2795\(75\)90109-9](https://doi.org/10.1016/0005-2795(75)90109-9).
- Natarajan, Nagarajan, Inderjit S. Dhillon, Pradeep Ravikumar, and Ambuj Tewari. 2013. "Learning with Noisy Labels." *Advances in Neural Information Processing Systems (NIPS)*.
- Neto, Edmilson Campos, Daniel Alencar da Costa, and Uirá Kulesza. 2018. "The Impact of Refactoring Changes on the SZZ Algorithm: An Empirical Study." *Proceedings of the IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 380–90.
- Ni, Chao, Wei Wang, Kaiwen Yang, Harald Gall, and Xiang Liu. 2022. "The Best of Both Worlds: Integrating Semantic Features with Expert Features for Defect Prediction and Localization." *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*.
- Northcutt, Curtis G., Lu Jiang, and Isaac L. Chuang. 2021. "Confident Learning: Estimating Uncertainty in Dataset Labels." *Journal of Artificial Intelligence Research* 70: 1373–411. <https://doi.org/10.1613/jair.1.12125>.
- Oza, Nikunj C., and Stuart Russell. 2001. "Online Bagging and Boosting." *Proceedings of the Eighth International Workshop on Artificial Intelligence and Statistics (AISTATS)*, PMLR, vol. R3: 229–36.
- Pornprasit, Chanathip, and Chakkrit Tantithamthavorn. 2021. "JITLine: A Simpler, Better, Faster, Finer-Grained Just-in-Time Defect Prediction." *Proceedings of the 18th International Conference on Mining Software Repositories (MSR)*, 369–79.
- Rosa, Giovanni, Luca Pascarella, Simone Scalabrino, et al. 2021. "Evaluating SZZ Implementations Through a Developer-Informed Oracle." *Proceedings of the 43rd IEEE/ACM International Conference on Software*

Engineering (ICSE). <https://doi.org/10.1109/ICSE43902.2021.00049>.

Rosa, Giovanni, Luca Pascarella, Simone Scalabrino, et al. 2023. “A Comprehensive Evaluation of SZZ Variants Through a Developer-Informed Oracle.” *Journal of Systems and Software* 202: 111729.

<https://doi.org/10.1016/j.jss.2023.111729>.

Śliwerski, Jacek, Thomas Zimmermann, and Andreas Zeller. 2005. “When Do Changes Induce Fixes?” *Proceedings of the 2005 International Workshop on Mining Software Repositories (MSR)*, 1–5.

<https://doi.org/10.1145/1083142.1083147>.

Wang, Shuo, Leandro L. Minku, and Xin Yao. 2015. “Resampling-Based Ensemble Methods for Online Class Imbalance Learning.” *IEEE Transactions on Knowledge and Data Engineering* 27 (5): 1356–68.

Wilcoxon, Frank. 1945. “Individual Comparisons by Ranking Methods.” *Biometrics Bulletin* 1 (6): 80–83.

Zeng, Zhengran, Yuqun Zhang, Haotian Zhang, and Lingming Zhang. 2021. “Deep Just-in-Time Defect Prediction: How Far Are We?” *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. <https://doi.org/10.1145/3460319.3464819>.

Zhao, Yunhua, Kostadin Damevski, and Hui Chen. 2023. “A Systematic Survey of Just-in-Time Software Defect Prediction.” *ACM Computing Surveys* 55 (10): 1–35. <https://doi.org/10.1145/3567550>.